UNIVERSIDAD DE SANTIAGO DE CHILE
FACULTAD DE INGENIERÍA
Departamento de Ingeniería Informática



Actualización del núcleo tecnológico de EDDIE

Nicolás Alonso Muñoz Reyes

Profesor guía: Roberto González Ibáñez

Tesis de grado presentada en
conformidad a los requisitos
para obtener el título de Ingeniero
Civil en Informática

Santiago – Chile
2026

© **Nicolás Alonso Muñoz Reyes** , 2026

 • Algunos derechos reservados. Esta obra está bajo una Licencia Creative Commons Atribución-Chile 3.0. Sus condiciones de uso pueden ser revisadas en: `http://creativecommons.org/licenses/by/3.0/cl/`.

RESUMEN

EDDIE es una plataforma de investigación del laboratorio InTeractiOn de la Universidad de Santiago de Chile que habilita la lectura aumentada no obstructiva sobre documentos impresos: un proyector y dos cámaras sobre el escritorio permiten resaltar una palabra en el papel y obtener su definición, traducción o contenido relacionado, sincronizar anotaciones entre el libro y su versión PDF y registrar la mirada del lector con un eye tracker. Tras seis memorias de título sucesivas, el sistema llegó a unas 45 000 líneas de C# en 26 proyectos, con tres mecanismos distintos de carga de módulos por reflexión y 67 errores documentados por Ibaceta Jaña (2023), lo que impedía usarlo de forma confiable como instrumento de medición.

Este trabajo reconstruye el núcleo de EDDIE en Python bajo una arquitectura de plugins con un contrato único (`PluginBase`), un orquestador central (`OrchestratorCore`) que ejecuta un ciclo de inicialización, ejecución y liberación con aislamiento de fallos, y una capa de abstracción de hardware (`DeviceFactory`) que conmuta cada dispositivo entre su implementación real y una simulada. Siguiendo el ciclo PDCA con prototipado rápido (RAD) y Kanban, se evaluó primero la factibilidad de migrar 19 funciones críticas mediante prototipos desechables y luego se integraron, en siete iteraciones, los módulos de consistencia físico-digital, reconocimiento de texto, interacción gestual, rastreo ocular, búsqueda (respondida por un modelo de lenguaje en lugar de las APIs vencidas) y voz, con una interfaz rediseñada a partir de los hallazgos de usabilidad de la evaluación previa.

La versión final aprueba 107 pruebas automatizadas con una cobertura de líneas de 81% y una latencia del ciclo completo del orquestador de 1,48 ms en el percentil 95, bajo el umbral de 8,3 ms derivado de la frecuencia de los eye trackers. La verificación funcional con MECIVA PS y las pruebas de aceptación con el hardware del laboratorio completan la evaluación.

Palabras Claves: lectura aumentada; arquitectura de plugins; software científico; deuda técnica; rastreo ocular; Python

ABSTRACT

EDDIE is a research platform from the InTeractiOn laboratory at Universidad de Santiago de Chile that enables non-obstructive augmented reading on printed documents: a projector and two cameras above the desk let readers highlight a word on paper and obtain its definition, translation or related content, synchronize annotations between the book and its PDF, and record their gaze with an eye tracker. After six successive undergraduate theses, the system had grown to about 45,000 lines of C# in 26 projects, with three different reflection-based module loading mechanisms and 67 documented errors (Ibaceta Jaña, 2023), which prevented its reliable use as a measurement instrument.

This work rebuilds the EDDIE core in Python on a plugin architecture with a single contract (`PluginBase`), a central orchestrator that runs an initialize–execute–release cycle with fault isolation, and a hardware abstraction layer that switches each device between its real and simulated implementation. Following PDCA with rapid application development and Kanban, the feasibility of migrating 19 critical functions was first assessed with throwaway prototypes; the consistency, text recognition, gesture, eye tracking, search (answered by a language model instead of the expired web APIs) and voice modules were then integrated in seven iterations, with a user interface redesigned from previous usability findings.

The final version passes 107 automated tests with 81% line coverage and a 95th-percentile orchestrator cycle latency of 1,48 ms, below the 8.3 ms threshold derived from the eye trackers' sampling rate. Functional verification with MECIVA PS and acceptance tests with the laboratory hardware complete the evaluation.

Keywords: augmented reading; plugin architecture; research software; technical debt; eye tracking; Python

Dedicado a...

AGRADECIMIENTOS

Agradezco a

TABLA DE CONTENIDO

1 Introducción **1**
- 1.1 Antecedentes y motivación . . . 1
- 1.2 Descripción del problema . . . 2
- 1.3 Solución propuesta . . . 2
- 1.4 Objetivos y alcance del proyecto . . . 2
  - 1.4.1 Objetivo general . . . 2
  - 1.4.2 Objetivos específicos . . . 3
  - 1.4.3 Alcances y limitaciones . . . 3
- 1.5 Metodología y herramientas utilizadas . . . 4
  - 1.5.1 Metodología . . . 4
  - 1.5.2 Herramientas de desarrollo . . . 5
  - 1.5.3 Ambiente de desarrollo . . . 5
- 1.6 Organización del documento . . . 6

2 Marco teorico **7**
- 2.1 Marco Conceptual . . . 7
- 2.2 Estado del arte . . . 7
  - 2.2.1 Antecedentes del Proyecto (Evolución Histórica) . . . 7
    - 2.2.1.1 Catálogo de funcionalidades y levantamiento de la usabilidad previa 8
  - 2.2.2 Sistemas de lectura aumentada similares . . . 9
  - 2.2.3 El problema de la modernización en software de investigación . . . 12
  - 2.2.4 Multimodal Observer (MO) como referente arquitectural . . . 13

3 Análisis **14**
- 3.1 Diagnóstico . . . 14
- 3.2 Captura de requerimientos . . . 14
  - 3.2.1 Fuentes y etapas del proceso . . . 14
  - 3.2.2 Requerimientos funcionales . . . 15
  - 3.2.3 Requerimientos no funcionales . . . 16
- 3.3 Factibilidad técnica . . . 18
- 3.4 Prototipado . . . 18
  - 3.4.1 Síntesis de la selección de funciones . . . 25
  - 3.4.2 Prototipo F1: Enumeración e Inicialización de Cámaras . . . 25
  - 3.4.3 Prototipo F2: Preprocesamiento y Binarización de Texto . . . 27
  - 3.4.4 Prototipo F3: Reconocimiento Óptico de Caracteres . . . 28
  - 3.4.5 Prototipo F4: Detección Automática de Páginas . . . 29
  - 3.4.6 Prototipo F5: Seguimiento de Puntero de Color . . . 30
  - 3.4.7 Prototipo F6: Segmentación de Mano y Apuntado . . . 32
  - 3.4.8 Prototipo F7: Mapeo de Coordenadas y Calibración Matricial . . . 33
  - 3.4.9 Prototipo F9: Entrada por Mouse como Alternativa . . . 34
  - 3.4.10 Prototipo F10: Sockets de Comunicación con el Rastreador Ocular . . . 35
  - 3.4.11 Prototipo F11: Aislamiento y Carga Dinámica de Plugins de Rastreo Ocular . 36
  - 3.4.12 Prototipo F12: Detección de Fijación Atencional . . . 37
  - 3.4.13 Prototipo F13: Renderizado del Retículo de Mirada . . . 38
  - 3.4.14 Prototipo F14: Búsqueda de Coordenadas QuadPoints . . . 39
  - 3.4.15 Prototipo F15: Sincronización de Destacado . . . 40
  - 3.4.16 Prototipo F16: Sincronización de Notas Adhesivas . . . 41
  - 3.4.17 Prototipo F17: Sincronización de Figuras Geométricas . . . 42
  - 3.4.18 Prototipo F24: Reconocimiento de Comandos de Voz . . . 43
  - 3.4.19 Prototipo F25: Síntesis de Voz . . . 45

3.4.20 Prototipo F27: Herramienta de Subrayado Digital Proyectado 46

4 Diseño y aspectos de implementación 48
4.1 Arquitectura general 48
4.2 Contratos de plugin 49
4.3 Núcleo orquestador 50
4.3.1 Ciclo de vida 50
4.3.2 Configuración declarativa 51
4.3.3 Capa de abstracción de hardware 51
4.3.4 Integración con la interfaz 51
4.4 Módulos integrados 52
4.5 Diseño de la interfaz 53
4.6 Iteraciones de desarrollo 55

5 Evaluación 56
5.1 Plan de evaluación 56
5.2 Pruebas de software 57
5.2.1 Pruebas automatizadas y cobertura 57
5.2.2 Latencia del orquestador 57
5.2.3 Aislamiento de fallos 58
5.2.4 Matriz de trazabilidad 58
5.2.5 Hallazgos de la verificación 59
5.3 Verificación y validación con MECIVA PS 60
5.3.1 Protocolo 60
5.3.2 Resultados 61
5.4 Pruebas de aceptación 61
5.5 Síntesis de la evaluación 62

6 Conclusiones 63
6.1 Cumplimiento de los objetivos 63
6.2 Respuesta a la pregunta de investigación 64
6.3 Limitaciones 64
6.4 Trabajo futuro 65

Glosario 66

Referencias bibliográficas 67

Anexos 68

A Errores de EDDIE-2023 y plantilla de trazabilidad 68

B Plan de trabajo 69

Apéndices 70

A Fichas de los avances 70

B Mediciones de los prototipos 74

C Comparación de la interfaz 75

D Manual de instalación y uso 76
D.1 Instalación 76
D.2 Ejecución 76
D.3 Uso 76

ÍNDICE DE TABLAS

Tabla 2.1 Síntesis de la evolución histórica del sistema EDDIE a través de seis memorias de título sucesivas. 8
Tabla 2.2 Comparación de sistemas. 12

Tabla 3.1 Resumen de requerimientos funcionales por módulo. 16
Tabla 3.2 Requerimientos no funcionales. 17
Tabla 3.3 Funciones del subsistema de procesamiento de imágenes y visión computacional. 19
Tabla 3.4 Funciones del subsistema de reconocimiento gestual e interacción sin contacto. 20
Tabla 3.5 Funciones del subsistema de rastreo ocular y atención visual (Eye Tracking). . 21
Tabla 3.6 Funciones del subsistema de consistencia físico-digital en PDFs. 22
Tabla 3.7 Funciones del subsistema de búsqueda web de conocimiento y APIs REST. . . 23
Tabla 3.8 Funciones del subsistema de voz, audio e intenciones. 24
Tabla 3.9 Funciones del subsistema de presentación GUI, proyección y logs. 24
Tabla 3.10 Ficha de validación del prototipo: Enumeración e Inicialización de Cámaras. . 26
Tabla 3.11 Ficha de validación del prototipo: Preprocesamiento y Binarización de Texto. . 27
Tabla 3.12 Ficha de validación del prototipo: Reconocimiento Óptico de Caracteres. . . . 28
Tabla 3.13 Ficha de validación del prototipo: Detección Automática de Páginas. 30
Tabla 3.14 Ficha de validación del prototipo: Seguimiento de Puntero de Color. 31
Tabla 3.15 Ficha de validación del prototipo: Segmentación de Mano y Apuntado. 32
Tabla 3.16 Ficha de validación del prototipo: Mapeo de Coordenadas y Calibración Matricial. 33
Tabla 3.17 Ficha de validación del prototipo: Entrada por Mouse como Alternativa. 34
Tabla 3.18 Ficha de validación del prototipo: Sockets de Comunicación con el Rastreador Ocular. 35
Tabla 3.19 Ficha de validación del prototipo: Aislamiento y Carga Dinámica de Plugins de Rastreo Ocular. 36
Tabla 3.20 Ficha de validación del prototipo: Detección de Fijación Atencional. 37
Tabla 3.21 Ficha de validación del prototipo: Renderizado del Retículo de Mirada. 38
Tabla 3.22 Ficha de validación del prototipo: Búsqueda de Coordenadas QuadPoints. . . 39
Tabla 3.23 Ficha de validación del prototipo: Sincronización de Destacado. 40
Tabla 3.24 Ficha de validación del prototipo: Sincronización de Notas Adhesivas. 42
Tabla 3.25 Ficha de validación del prototipo: Sincronización de Figuras Geométricas. . . . 43
Tabla 3.26 Ficha de validación del prototipo: Reconocimiento de Comandos de Voz. . . . 44
Tabla 3.27 Ficha de validación del prototipo: Síntesis de Voz. 45
Tabla 3.28 Ficha de validación del prototipo: Herramienta de Subrayado Digital Proyectado. 46

Tabla 4.1 Contratos de plugin y sus implementaciones. 49
Tabla 4.2 Avances del desarrollo y resultados de sus pruebas. 55

Tabla 5.1 Métricas de evaluación, criterio de aceptación y estado. 56
Tabla 5.2 Latencia por plugin en 500 ciclos del orquestador (ms). 57
Tabla 5.3 Matriz de trazabilidad (parcial) de errores de EDDIE-2023. 59
Tabla 5.4 Hallazgos de la verificación y su corrección. 60
Tabla 5.5 Escala de evaluación de las pruebas de aceptación. 61
Tabla 5.6 Pruebas de aceptación planificadas. 62

Tabla A.1 Extracto de errores encontrados en EDDIE-2023. 68
Tabla A.2 Plantilla de la matriz de trazabilidad. 68

Tabla B.1 Plan de trabajo. 69

Tabla A.1 Primer prototipo - Núcleo orquestador y contratos de plugins. 70
Tabla A.2 Segundo prototipo - Capa de abstracción de hardware y módulo de consistencia. 70
Tabla A.3 Tercer prototipo - Reconocimiento de texto e interfaz de configuración. 71
Tabla A.4 Cuarto prototipo - Interacción gestual. 71
Tabla A.5 Quinto prototipo - Rastreo ocular. 72
Tabla A.6 Sexto prototipo - Pantalla de proyección, búsqueda con LLM y voz. 72
Tabla A.7 Séptimo prototipo - Versión final verificada. 73
Tabla A.8 Hallazgos de la verificación. 73

Tabla B.1 Resumen de las mediciones de los prototipos. 74

ÍNDICE DE ILUSTRACIONES

Figura 2.1 El primer prototipo de DigitalDesk. 9
Figura 2.2 Gesto de señalamiento del usuario para EnhancedDesk. 10
Figura 2.3 Descripción general de AR-PaperSync. 11
Figura 2.4 Enlight: Lectura Aumentada. 11
Figura 2.5 Multimodal Observer (MO) - Vista principal. 13

Figura 4.1 Arquitectura en capas de EDDIE en Python. 48
Figura 4.2 Pantalla de inicio con los pasos de preparación y el estado de cada módulo. 54
Figura 4.3 Configuración del reconocimiento de texto (izquierda) y de la interacción gestual con lápiz (derecha). 54
Figura 4.4 Configuración del documento PDF (izquierda) y del rastreo ocular (derecha). 54
Figura 4.5 Pantalla proyectada: controles en la franja superior (izquierda) y resultado de una búsqueda (derecha). 55

Figura C.1 Pantalla de inicio: EDDIE-2023 (izquierda) y versión en Python (derecha). . 75
Figura C.2 Configuración de gestos: EDDIE-2023 (izquierda) y versión en Python (derecha). 75
Figura C.3 Pantalla proyectada: EDDIE-2023 (izquierda) y versión en Python (derecha). 75

ÍNDICE DE ALGORITMOS

Algoritmo 4.1 Ciclo de vida del orquestador. 50

CAPÍTULO 1. INTRODUCCIÓN

1.1 ANTECEDENTES Y MOTIVACIÓN

La lectura constituye un proceso cognitivo fundamental para el aprendizaje y la construcción de conocimiento, el cual ha experimentado una transformación sustancial con la digitalización de la información. En el ámbito de la Interacción Humano-Información (HII), resulta de especial interés analizar cómo las personas interactúan con distintos soportes de lectura y de qué manera estas modalidades impactan en factores como la comprensión lectora, la retención de ideas y la carga cognitiva. A pesar de la masificación de los dispositivos electrónicos, la evidencia empírica muestra que el material impreso conserva vigencia y sigue siendo preferido para tareas de lectura profunda (Saleh & Mashhur, 2015; Singer & Alexander, 2016, como se citaron en Santibáñez Espinoza, 2020).

Para avanzar en el estudio sistemático de estos fenómenos, la comunidad científica requiere de herramientas y metodologías experimentales rigurosas. La investigación en este campo depende de entornos que integren adecuadamente componentes de hardware, software y diseños experimentales capaces de capturar y evaluar, de manera precisa y no intrusiva, las dinámicas de interacción del lector durante el proceso de lectura. Dentro de las aproximaciones tecnológicas en esta área, una de las líneas menos exploradas y estudiadas es la lectura aumentada aplicada sobre papel físico. En este contexto, la lectura aumentada no se refiere a la superposición de animaciones recreativas o elementos tridimensionales lúdicos, sino al enriquecimiento del material impreso mediante prestaciones funcionales equivalentes a las del formato digital (tales como búsquedas en línea de términos dentro del texto, traducción instantánea, consulta de definiciones en contexto, despliegue de contenido complementario y sincronización bidireccional de anotaciones entre el documento físico y su versión digital).

Las herramientas capaces de soportar este tipo de experiencias integradas son escasas tanto en la literatura como en la práctica experimental. En el contexto local de investigación del Departamento de Ingeniería Informática de la Universidad de Santiago de Chile (USACH), y bajo la línea de trabajo del grupo InTeractiOn, nació la plataforma EDDIE (Empowering Digital Paper with Interactive Enhancements), concebida precisamente como un instrumento de investigación para habilitar experiencias de lectura aumentada no obstructiva sobre documentos impresos (Carrasco Madriaga, 2017). A lo largo de los años, EDDIE ha experimentado múltiples fases de desarrollo e integración incremental a través de aportes de distintos memoristas e investigadores de la unidad académica, quienes buscaron incorporar capacidades de procesamiento visual y seguimiento (Ibaceta Jaña, 2023).

Sin embargo, debido al tiempo transcurrido desde su concepción original y a la acumulación de evoluciones heterogéneas desarrolladas en diferentes etapas, la herramienta actualmente presenta limitaciones y desactualizaciones operativas. Estas condiciones impiden que EDDIE pueda ser utilizada de manera plena, estable y confiable en estudios experimentales contemporáneos sobre comprensión y comportamiento lector, limitando su potencial como plataforma de medición científica.

1.2 DESCRIPCIÓN DEL PROBLEMA

El problema central del proyecto EDDIE no radica en el volumen total de errores documentados, sino en un riesgo arquitectónico específico sobre una base de aproximadamente 45.000 líneas de código distribuidas en 26 proyectos. De los 67 problemas registrados por Ibaceta Jaña (2023), su reclasificación por causa (Tabla 3.1 del Anexo) muestra que 16 (23%) corresponden a la infraestructura técnica de carga y despliegue, mientras que el resto se distribuye entre errores de práctica de código, hardware/sensores, servicios externos, UI/idioma, infraestructura de laboratorio y documentación. Aunque los 4 errores de carga por reflexión representan sólo una fracción de esta infraestructura técnica, tienen un impacto desproporcionado: cada uno impide el arranque completo de un módulo entero, y su causa raíz constituye un riesgo estructural recurrente: cualquier desarrollador que integre un nuevo plugin puede reproducir el mismo tipo de falla, dado que no existe una única interfaz validada que lo prevenga. Ante este escenario, ¿En qué medida la reconstrucción del núcleo de EDDIE bajo una arquitectura de plugins con contrato único permite reducir el número de errores estructurales documentados y habilitar su uso como instrumento de medición científica confiable?

1.3 SOLUCIÓN PROPUESTA

1.4 OBJETIVOS Y ALCANCE DEL PROYECTO

1.4.1 Objetivo general

Actualizar el núcleo tecnológico del proyecto EDDIE mediante el rediseño y reimplementación en Python e integración de los módulos de investigación existentes, para reducir la

deuda técnica y asegurar la compatibilidad con sensores físicos del proyecto en un entorno de lectura aumentada no obstructiva.

1.4.2 Objetivos específicos

1. Evaluar mediante prototipos desechables en Python la factibilidad técnica de migrar los módulos existentes a las tecnologías propuestas.
2. Diseñar el núcleo orquestador de EDDIE, definiendo los contratos de interfaz que permitan la integración de los módulos previos como plugins desacoplados.
3. Implementar el núcleo orquestador en Python, junto con la capa de abstracción de sensores, integrando los módulos de consistencia bidireccional, reconocimiento de texto, interacción gestual y rastreo ocular bajo la nueva arquitectura.
4. Validar el sistema integrado mediante pruebas de software, verificación funcional y pruebas de aceptación en el ambiente operativo real del laboratorio InTeractiOn.

1.4.3 Alcances y limitaciones

La solución cubre la reconstrucción completa del núcleo orquestador, los cinco contratos de interfaz de plugin (sección 3.2) y la capa de abstracción de hardware (DeviceFactory) con conmutación real/simulado. La validación con hardware real se limita a los cuatro módulos del MVP (IConsistencyProvider, IEyeTracker sobre GazePoint/TheEyeTribe, IImageProcessor e IGesturePlugin) en el ambiente del laboratorio InTeractiOn. La implementación funcional de IWebSearchProvider (queda solo como interfaz), dashboard web, visor de proyección en navegador, CLI de validación y su canal de telemetría WebSocket, reconocimiento automático del libro físico y catálogo de lectura, eliminación de accesorios mediadores (mouse, lápices de color) y recalibración de visión computacional más allá de la mínima necesaria para el MVP. La migración de Emgu CV a OpenCV no es una traducción directa (ambas bibliotecas producen resultados numéricos distintos ante los mismos parámetros), por lo que los algoritmos de reconocimiento se deben recalibrar y validar en el nuevo entorno. Al ejecutarse todos los plugins como objetos en memoria dentro del proceso del OrchestratorCore, se elimina el overhead de red, pero el aislamiento de fallos depende del manejo de excepciones centralizado (sección 3.2). La proyección se renderiza en el mismo proceso mediante ventana nativa, el WebSocket queda

reservado para telemetría externa, fuera de este alcance. Además se eliminan las dependencias nativas de 32 bits mejorando la portabilidad potencial del software, pero no está verificada el desarrollo es exclusivamente en Windows 11 y depende de SDKs de fabricante específicos de plataforma. La verificación en Linux/macOS queda fuera de alcance, y en cualquier caso esta mejora no implica portabilidad del sistema completo, atado a un montaje físico particular.

1.5 METODOLOGÍA Y HERRAMIENTAS UTILIZADAS

1.5.1 Metodología

Para la modernización del núcleo de EDDIE se emplea el ciclo PDCA (Plan-Do-Check-Act), complementado con Kanban para el seguimiento de tareas y Rapid Application Development (RAD) para el desarrollo iterativo mediante prototipos. Esta combinación replica el enfoque validado por Figueroa (2025) en la reimplementación de MO, dada la similitud estructural entre ambos proyectos. El ciclo PDCA estructura el trabajo en cuatro etapas, correspondientes a los objetivos específicos: Planificación (Plan), con el diagnóstico del estado actual de EDDIE y la identificación de los nudos críticos de incompatibilidad (OE1). Ejecución (Do), con el diseño e implementación del núcleo orquestador, la capa de abstracción de sensores y la integración de los módulos existentes como plugins mediante iteraciones sucesivas de prototipado bajo RAD (OE2 y OE3). Verificación (Check), con las pruebas de software, verificación funcional y pruebas de aceptación descritas en la sección de evaluación. Y Actuación (Act), con la incorporación de ajustes derivados de dichas pruebas y el cierre de la documentación técnica.

Un aspecto central de RAD es el uso de prototipos desechables (throwaway prototyping) durante las primeras semanas de la etapa de ejecución, en lugar de incorporar directamente cada funcionalidad migrada al código definitivo. Cada componente crítico se valida primero mediante un prototipo aislado y de bajo costo, cuyo único propósito es confirmar la factibilidad técnica de la migración a Python antes de invertir tiempo en su integración formal al orquestador. Este criterio, aplicado exitosamente por Figueroa Contreras (2025) en MO, permite detectar tempranamente bloqueos de compatibilidad y evita destinar semanas de desarrollo a funcionalidades que podrían requerir un enfoque alternativo. Kanban se utiliza para visualizar el flujo de tareas de cada módulo y fase del ciclo PDCA a lo largo de las 17 semanas del proyecto.

1.5.2 Herramientas de desarrollo

Las herramientas empleadas durante el desarrollo del proyecto son las siguientes:

- Python, lenguaje principal para la implementación del núcleo orquestador, la capa de abstracción de sensores y los plugins.
- OpenCV (binding para Python), utilizado para las funcionalidades de visión computacional que en la versión original de EDDIE dependían de Emgu CV.
- pytest, para el desarrollo de pruebas unitarias y de integración.
- SonarQube, para el análisis de calidad del código durante las pruebas de software.
- Git y GitHub, para el control de versiones y la gestión del repositorio del proyecto.
- Visual Studio Code, como entorno de desarrollo integrado.
- EDDIE (versión actual en C#), utilizado como referencia funcional durante el diagnóstico y como línea base para la comparación de resultados.
- Overleaf (LaTeX), para la redacción del informe de título.
- Notion, para la documentación técnica, el seguimiento de tareas mediante Kanban y el registro de avances durante el desarrollo.

1.5.3 Ambiente de desarrollo

El desarrollo del proyecto se realiza en dos ambientes complementarios. El primero corresponde a un equipo de trabajo personal utilizado para la implementación, pruebas unitarias y de integración del núcleo orquestador y los plugins, el cual cuenta con las siguientes características:

- Procesador AMD Ryzen 5 7600, 6 núcleos, 3.80 GHz
- Memoria RAM de 32 GB (5200 MT/s)
- Tarjeta gráfica dedicada NVIDIA GeForce RTX 5060 (8 GB VRAM) y GPU integrada AMD Radeon
- Almacenamiento SSD de 932 GB

- Sistema operativo Windows 11 Pro (versión 25H2)

En este ambiente se realiza la mayor parte del ciclo de desarrollo iterativo bajo RAD, incluyendo la implementación del orquestador, la integración de los plugins de software y las pruebas descritas en la etapa de verificación del ciclo PDCA.

El segundo ambiente corresponde a la sala de experimentación del Departamento de Ingeniería Informática de la USACH, donde se encuentra el despliegue físico de EDDIE.

Este ambiente cuenta con el computador designado para la operación de EDDIE, dos cámaras digitales utilizadas para el reconocimiento de texto y el reconocimiento gestual, un proyector actúa como display sobre el escritorio, y dispositivos de rastreo ocular como GazePoint y TheEyeTribe. Este ambiente se utiliza específicamente para la etapa de verificación del ciclo PDCA, donde se ejecutan las pruebas de aceptación y la validación del sistema con el hardware físico real, resolviendo así la dependencia de sensores virtuales documentada en etapas previas del proyecto.

1.6 ORGANIZACIÓN DEL DOCUMENTO

CAPÍTULO 2. MARCO TEORICO

2.1 MARCO CONCEPTUAL

2.2 ESTADO DEL ARTE

2.2.1 Antecedentes del Proyecto (Evolución Histórica)

Para comprender los fundamentos sobre los que se sustenta la presente propuesta de modernización, es necesario reconstruir la trayectoria histórica del sistema EDDIE (Entorno Digital y Dinámico de Interacción Escrita), desarrollado de forma incremental en el Departamento de Ingeniería Informática de la Universidad de Santiago de Chile bajo el alero del laboratorio InTeractiOn. El proyecto constituye un caso representativo de software científico que ha evolucionado orgánicamente a través de seis memorias de título sucesivas, cada una incorporando un nuevo módulo funcional sin que existiera, en ningún momento del proceso, una revisión sistemática de la arquitectura común subyacente.

El concepto original fue propuesto por Carrasco Madriaga (2017), quien implementó un entorno de realidad aumentada móvil orientado a enriquecer la lectura en papel mediante dispositivos portátiles. Contreras Velásquez (2019) evolucionó esta propuesta hacia una configuración no obstructiva de escritorio, sustituyendo el dispositivo móvil por un sistema fijo de cámara y proyector que elimina la necesidad de mediadores físicos manuales. A partir de esta base, el proyecto se expandió mediante la incorporación sucesiva de tres capacidades de soporte al lector: Montanares Mundaca (2019) integró un módulo de búsqueda web contextual activado por voz y gestos; Pezoa Riquelme (2020) desarrolló un mecanismo de consistencia bidireccional entre el documento físico y su versión digital en PDF; y Estay Vera (2020b) incorporó dispositivos de rastreo ocular (eye-tracking) para el análisis del comportamiento atencional del usuario. Posteriormente, Santibáñez Espinoza (2020) realizó un rediseño de la interfaz gráfica bajo un enfoque centrado en el usuario, y finalmente Ibaceta Jaña (2023) aplicó la metodología MECIVA PS para diagnosticar cuantitativamente el estado del software, identificando 67 errores estructurales acumulados a lo largo de este proceso de crecimiento.

Este proceso incremental produjo un sistema monolítico de aproximadamente 45.000 líneas de código distribuidas en 26 proyectos .csproj, en el cual la falta de una arquitectura común provocó la coexistencia de tres mecanismos heterogéneos de carga por reflexión (Assembly.LoadFrom, AppDomain Proxy Sandbox y Assembly.LoadFile), origen directo de más

Tabla 2.1: Síntesis de la evolución histórica del sistema EDDIE a través de seis memorias de título sucesivas.

Trabajo / Investigador	**Aporte al proyecto EDDIE**	**Limitaciones / Deuda técnica generada**
Carrasco Madriaga (2017)	Prototipo inicial de lectura aumentada en dispositivos móviles.	Dependencia de mediadores manuales (pantalla móvil).
Contreras Velásquez (2019)	Configuración no obstructiva de escritorio (cámara + proyector).	Acoplamiento directo con la interfaz WinForms en C#.
Montanares Mundaca (2019)	Módulo de búsqueda web contextual (Wikipedia, diccionarios).	Carga mediante reflexión con `Assembly.LoadFile`.
Pezoa Riquelme (2020)	Módulo de consistencia bidireccional físico-digital (PDF).	Incompatibilidades de sincronización y fallo por rutas fijas.
Estay Vera (2020b)	Integración de rastreo ocular (eye-tracking).	Carga aislada por Proxy Sandbox en `AppDomain` subordinado.
Santibáñez Espinoza (2020)	Rediseño centrado en el usuario de la interfaz gráfica.	Mantenimiento de la capa monolítica subyacente.
Ibaceta Jaña (2023)	Diagnóstico cuantitativo de errores mediante metodología MECIVA PS.	Revelación de 67 errores estructurales por falta de validación en hardware real.

Fuente: Elaboración propia a partir de Carrasco Madriaga (2017), Contreras Velásquez (2019), Montanares Mundaca (2019), Pezoa Riquelme (2020), Estay Vera (2020b), Santibáñez Espinoza (2020) e Ibaceta Jaña (2023).

de 30 de los 67 errores documentados por Ibaceta Jaña (2023).

2.2.1.1 Catálogo de funcionalidades y levantamiento de la usabilidad previa

En su estado actual, EDDIE ofrece cinco categorías funcionales heredadas directamente de las memorias descritas: procesamiento de imágenes y reconocimiento gestual (Contreras Velásquez, 2019), búsqueda web contextual (Montanares Mundaca, 2019), consistencia bidireccional entre el documento físico y su versión digital (Pezoa Riquelme, 2020), rastreo ocular (Estay Vera, 2020b), y una capa de presentación en WinForms resultante del rediseño centrado en el usuario de Santibáñez Espinoza (2020). Cada una de estas funcionalidades fue incorporada mediante un mecanismo de integración propio del memorista responsable, sin que se estableciera en ningún caso un contrato de interfaz común entre ellas, lo cual explica la heterogeneidad de mecanismos de carga identificada posteriormente en el diagnóstico del sistema.

El levantamiento de usabilidad realizado por Ibaceta Jaña (2023) mediante la metodología MECIVA PS constituye el diagnóstico más completo disponible sobre el estado funcional de EDDIE previo a esta propuesta. Su evaluación identificó fallas tanto internas (relacionadas con la consistencia de los módulos entre sí) como externas, vinculadas a la interacción del sistema con el hardware físico y a la ausencia de documentación de uso e instalación. Entre los hallazgos más críticos se cuentan fallas no resueltas en la conexión con hardware, como la inestabilidad de la cámara digital (error E16) y la imposibilidad de detectar

zonas de la proyección mediante el eyetracker (error E49), lo que compromete directamente la validez de EDDIE como instrumento de medición científica del comportamiento atencional. Este diagnóstico funcional constituye la línea base contra la cual se contrastaron los resultados de la reconstrucción propuesta en este trabajo.

2.2.2 Sistemas de lectura aumentada similares

El proyecto EDDIE no existe en un vacío tecnológico, por lo que existen tanto sistemas de lectura aumentada comparables, como plataformas de software que han enfrentado problemáticas de modernización arquitectónicas. Entre los referentes más destacados se encuentran:

- **DigitalDesk:** Este sistema pionero introdujo el concepto de ”Entorno Aumentado por Computadora”, utilizando una arquitectura basada en cámaras de video y un proyector electrónico montados sobre un escritorio físico convencional. Su objetivo es dotar a los documentos de papel de propiedades electrónicas y viceversa. Entre sus aplicaciones destacan una calculadora interactiva que permite al usuario señalar números impresos en un informe para ingresarlos automáticamente mediante OCR, un sistema de traducción que proyecta definiciones de palabras extranjeras directamente sobre el escritorio, y PaperPaint, una herramienta que permite seleccionar bocetos hechos a mano en papel y ”pegarlos” como objetos digitales proyectados.

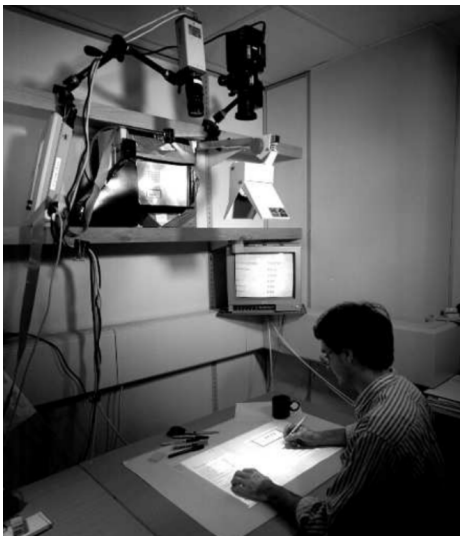


Figura 2.1: El primer prototipo de DigitalDesk.

Fuente: Wellner, P. (1993). Interacting with paper on the DigitalDesk. *Communications of the ACM*, 36(7), 87–96. https://doi.org/10.1145/159544.159630

- **EnhancedDesk:** Evoluciona la idea del escritorio aumentado resolviendo problemas críticos de visión artificial, como la sensibilidad a la iluminación. Arquitectónicamente, incorpora una cámara infrarroja que detecta el calor corporal para realizar un seguimiento de los dedos en tiempo real (25-30 cuadros por segundo) de manera robusta, sin importar los colores proyectados sobre la piel. Su aplicación principal es el ”Interactive Textbook”, donde códigos de matriz 2D en las páginas de un libro físico activan simulaciones digitales dinámicas (como experimentos de física de resortes o péndulos) que el estudiante puede manipular directamente con sus manos.

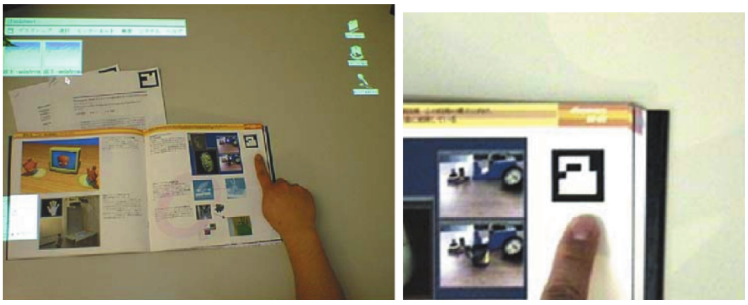


Figura 2.2: Gesto de señalamiento del usuario para EnhancedDesk.

Fuente: Koike, H., Sato, Y., & Kobayashi, Y. (2001). Integrating paper and digital information on EnhancedDesk: A method for real-time finger tracking on an augmented desk system. ACM Transactions on Computer-Human Interaction, 8(4), 307–322. https://doi.org/10.1145/504704.504706

- **AR-PaperSync:** Es un sistema de Realidad Aumentada móvil que busca la integración perfecta entre documentos científicos físicos y digitales. Permite a los investigadores escanear el título de un artículo impreso para visualizar instantáneamente capas de información con citas interactivas y referencias guardadas. Su arquitectura soporta la sincronización en tiempo real de notas de lectura entre dispositivos móviles, aplicaciones de escritorio y el papel físico, reduciendo significativamente la carga cognitiva y mejorando la eficiencia en la navegación de literatura académica.

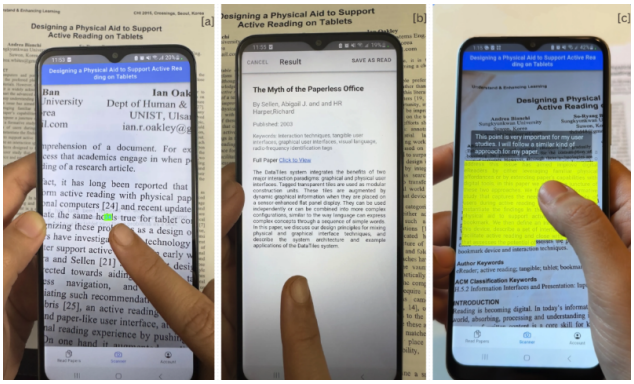


Figura 2.3: Descripción general de AR-PaperSync.

Fuente: agregar agregar agregar agregar agregar aaaaaaaaaaaaaaaaaaaggggggggggrrrrrreeeeeeeeggggggarrrr .
https://doi.org/10.1145/504704.504706

- **Enlight:** Es una lámpara de escritorio inteligente que utiliza visión computacional, reconocimiento óptico de caracteres, conexión a fuentes de datos en red y tecnología de projection-mapping para traer funciones inteligentes a libros impresos y documentos de cualquier época. Los usuarios pueden subrayar palabras para ver definiciones; si se resalta un pasaje, el sistema provee definiciones enciclopédicas contextuales, estadísticas de WolframAlpha y otros datos. Los highlights se sincronizan con la nube y pueden verse en otros dispositivos. A diferencia de EDDIE, es un prototipo conceptual de diseño de interacción (proyecto de grado MFA) sin módulos de eye-tracking ni consistencia bidireccional, pero comparte el mismo principio de proyección no obstructiva sobre el libro físico.

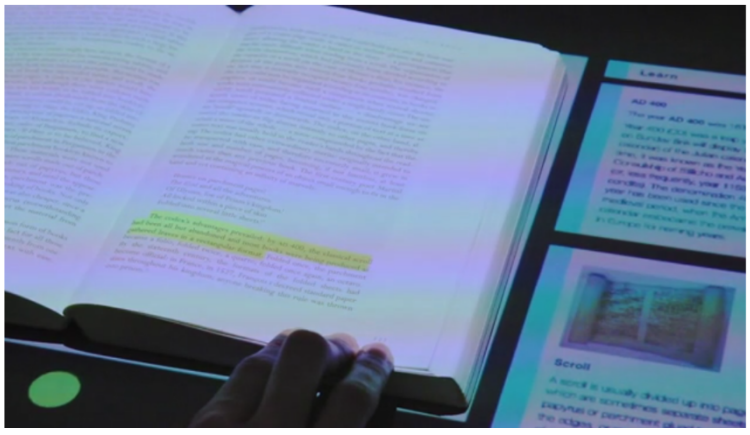


Figura 2.4: Enlight: Lectura Aumentada.

Fuente: . https://doi.org/10.1145/504704.504706

Tabla 2.2: Comparación de sistemas.

Sistema	Proyección sobre papel	Eye tracking	Consistencia bidireccional	Foco académico
EDDIE	Sí	Sí	Sí	Sí
DigitalDesk	Sí	No	Parcial	No
EnhancedDesk	Sí	No	No	Sí
AR-PaperSync	No (Móvil)	No	Sí	Sí
enlight	Sí	No	Parcial	Sí

Fuente: Elaboración propia.

2.2.3 El problema de la modernización en software de investigación

En este trabajo se entiende por software científico aquel desarrollado ad hoc para soportar una tarea de investigación específica, como la captura, procesamiento o análisis de datos experimentales, en contraposición al software de propósito general que un laboratorio pueda utilizar como herramienta auxiliar (sistemas operativos, editores de texto, suites ofimáticas). Bajo esta definición, EDDIE y MO son software científico, mientras que el sistema operativo o el editor de código empleados durante su desarrollo no lo son, dado que no fueron construidos para responder a un problema de investigación particular.

Un desafío documentado en el desarrollo de este tipo de software es que decisiones técnicas localmente óptimas (las que son tomadas por un investigador para resolver el problema inmediato de su propia tesis) tienden a resultar en un sistema globalmente inconsistente cuando se acumulan a través de múltiples autores sin una arquitectura común (Ernst et al., 2026). Este fenómeno ha sido documentado en sistemas de mayor escala, como plataformas de bioinformática y biología computacional, donde la falta de coordinación entre grupos de desarrollo genera herramientas difíciles de integrar entre sí (Orozco Solano, 2017). La literatura converge en que la solución recurrente a este problema es la adopción de arquitecturas modulares con interfaces estandarizadas que separan la lógica de investigación de la infraestructura técnica subyacente (Bustio Martínez et al., 2013; Glatard et al., 2017). En el dominio específico de los science gateways (plataformas que integran instrumentos y flujos de trabajo científicos), esta separación se implementa habitualmente mediante una capa de abstracción de hardware y un catálogo de plugins cargados dinámicamente a través de un único mecanismo, lo que permite aislar el fallo de un componente sin comprometer el resto del sistema (Glatard et al., 2017). Esta misma arquitectura de núcleo liviano con contratos de interfaz bien definidos es la que permite minimizar el tiempo dedicado a resolver problemas comunes de integración, concentrando el esfuerzo de desarrollo en la implementación de los algoritmos propios de cada dominio de investigación (Bustio Martínez et al., 2013).

2.2.4 Multimodal Observer (MO) como referente arquitectural

Un hito crítico para la propuesta actual es la evolución del MO, una aplicación del mismo laboratorio InTeractiOn diseñada para la captura y análisis de datos multimodales como movimientos oculares, bioseñales y video. MO enfrentó un problema estructuralmente idéntico al de EDDIE: las sucesivas versiones de su núcleo, desarrolladas por distintos tesistas para incorporar nuevos módulos, generaron incompatibilidades que dejaron la mayoría de sus plugins de visualización y análisis inoperativos en las versiones más recientes. La solución adoptada fue una re-implementación completa del núcleo en Python con una arquitectura de plugins desacoplada, la cual demostró ser viable en el contexto académico y produjo un sistema validado con métricas objetivas de calidad de código, rendimiento y aceptación (Figueroa Contreras, 2025). Esta experiencia constituye el antecedente empírico más relevante para la presente propuesta, tanto por la similitud estructural del problema como por compartir el mismo contexto institucional y línea de investigación.

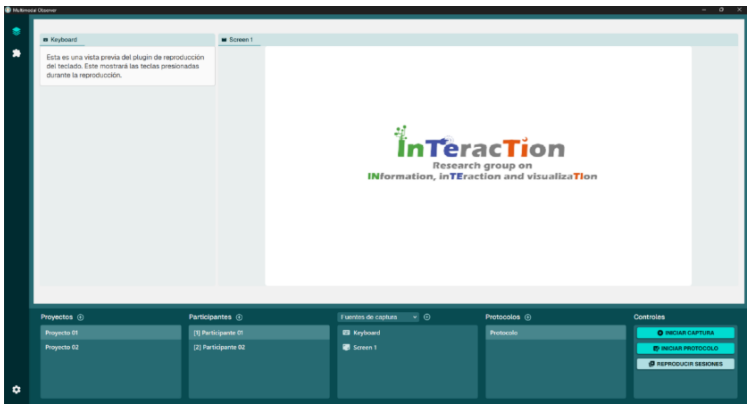


Figura 2.5: Multimodal Observer (MO) - Vista principal.

Fuente: Contreras, M. F. (2025). Arquitectura basada en plugins para la captura y análisis de datos multimodales: El caso de Multimodal Observer . Memoria de título, Universidad de Santiago de Chile.

CAPÍTULO 3. ANÁLISIS

3.1 DIAGNÓSTICO

3.2 CAPTURA DE REQUERIMIENTOS

La captura de requerimientos tuvo por objetivo definir qué debe hacer la nueva versión de EDDIE (augmEnteD reaDIng dEsk) y bajo qué condiciones de calidad debe hacerlo. El punto de partida fue el sistema legado, un desarrollo monolítico en C# ubicado en el repositorio EDDIE-2023. Dado que la actualización exige conocer tanto las capacidades que deben conservarse como las deficiencias que deben corregirse, el proceso se llevó a cabo mediante un enfoque multifacético y defensivo, con el fin de asegurar una cobertura completa y actualizada de las necesidades científicas y operativas del proyecto.

El proceso persiguió dos metas complementarias. La primera fue consolidar las funcionalidades de interacción físico-digital sobre papel impreso que el sistema ya ofrecía. La segunda fue resolver de manera estructural los 67 errores documentados por Ibaceta Jaña (2023), entre los que se cuentan variables con valor nulo, rutas locales de desarrolladores anteriores, y la falta de manuales de instalación y de uso. Ambas metas confluyen en una decisión arquitectónica: adoptar un diseño extensible basado en plugins sobre Python, línea inspirada en la evolución de Multimodal Observer (Figueroa Contreras, 2025).

Las fuentes de información y las etapas metodológicas empleadas se detallan a continuación.

3.2.1 Fuentes y etapas del proceso

Revisión de requerimientos e historial legado. Se analizaron los requerimientos funcionales y no funcionales del desarrollo original de la plataforma, compilados en los trabajos de titulación previos de EDDIE (Carrasco Madriaga, 2017; Contreras Velásquez, 2019; Montanares Mundaca, 2019; Pezoa Riquelme, 2020; Estay Vera, 2020b; Santibáñez Espinoza, 2020) y reunidos y diagnosticados por Ibaceta Jaña (2023). Esta revisión permitió establecer una línea base de las capacidades que debían mantenerse, como el trazado de destacados, la lectura de QuadPoints y el manejo de notas adhesivas, y distinguirlas de aquellas que requerían una reestructuración crítica, como la carga dinámica de ensamblados DLL y la gestión de punteros nulos.

Directrices del profesor guía y diagnóstico arquitectónico. Se integraron las prioridades

surgidas del diagnóstico de la arquitectura en C#, tomando como referencia las siete categorías de errores internos definidas por Ibaceta Jaña (2023). Como resultado, se estableció como requisito el desacoplamiento del núcleo respecto de bibliotecas de terceros del sistema legado, reemplazando iTextSharp por PyMuPDF y Emgu CV por OpenCV, y se incorporó un orquestador central que coordina los módulos.

Referentes de arquitectura y estado del arte. Como referente se consideró Multimodal Observer (MO), plataforma de captura y análisis multimodal desarrollada en el mismo laboratorio, InTeractiOn. El estado del arte de MO (Figueroa Contreras, 2025) describe herramientas como NoldusHub, OpenSignals y OBS Studio, y su diseño incorpora la sincronización temporal entre fuentes de captura, el registro de eventos en archivos JSON y la separación entre plugins de captura (Capture Plugins) y vistas de reproducción (Playback Views). Este referente es pertinente para EDDIE, ya que, este ya contaba con una arquitectura modular pensada para incorporar nuevos módulos (Contreras, 2019, requisitos RFCO12 y RFCO13 según Ibaceta Jaña (2023)) y con plugins por dispositivo de rastreo ocular, cargados como bibliotecas mediante la interfaz IEyeTracking (Estay Vera, 2020b). Segundo, de MO se adopta la idea de extensibilidad por plugins, que en EDDIE se generaliza desde el módulo de rastreo ocular hacia el resto de los módulos.

Estandarización y especificación de historias de usuario. Todos los requisitos recopilados se formalizaron y se clasificaron por módulos funcionales. Se eliminaron las ambigüedades y se redactó cada requisito de modo que fuese verificable, ya sea mediante pruebas unitarias con pytest o mediante los prototipos funcionales desarrollados bajo la metodología RAD.

3.2.2 Requerimientos funcionales

Los requerimientos funcionales describen las operaciones concretas que la plataforma debe realizar. La Tabla 3.1 presenta un resumen de los 55 requerimientos identificados, agrupados en siete módulos: Núcleo, Organización, Captura, Consistencia Físico-Digital, Visualización y los plugins de captura y de visualización. Para cada módulo se indica la cantidad de requerimientos y un conjunto de ejemplos representativos.

Tabla 3.1: Resumen de requerimientos funcionales por módulo.

Módulo	**Cantidad de RF**	**Ejemplos de requerimientos representativos**
Organización y Gestión Experimental	12	Crear, editar, eliminar, visualizar y proteger (bloqueo de solo lectura) proyectos de investigación, participantes y protocolos experimentales.
Núcleo y Orquestador Central	10	Administrar el ciclo de vida de plugins (initialize, execute, release), gestionar la persistencia en preferences.json, soportar el cambio dinámico de idioma (español/inglés) e integrar la bandeja del sistema (System Tray).
Captura de Datos y Sensores	9	Iniciar, pausar, reanudar y detener capturas síncronas de video de cámara y pantalla; registrar eventos de teclado y mouse en archivos JSON estructurados; gestionar sesiones de captura.
Consistencia Físico-Digital (F14–F17)	11	Búsqueda y mapeo de QuadPoints exactos en PDFs reales (F14); sincronización del destacado físico $\leftrightarrow$ digital; detección de post-its en papel con Tesseract OCR e inyección de anotaciones; clasificación de figuras geométricas mediante el índice de circularidad $C = 4\pi A/P^2$ y proyección de halos de luz.
Visualización y Reproducción Síncrona	7	Reproducir sesiones capturadas en una línea de tiempo unificada (Timeline Scrubber); sincronizar video, trazo del cursor del mouse y reconstrucción del texto del teclado; organizar espacios de trabajo en ventanas independientes.
Plugins de Captura y Hardware (HAL)	3	Extender conectores de hardware para rastreo ocular (EyeTracker), sensores gestuales y procesadores de imagen mediante contratos de interfaz estrictamente definidos (PluginBase).
Plugins de Visualización y Proyección	3	Desplegar overlays de proyección sobre el papel físico y vistas en miniatura de los canales de captura en tiempo real.
TOTAL	**55**	

Fuente: Elaboración propia, 2026.

La distribución de los requerimientos muestra que los módulos de Organización (12), Consistencia Físico-Digital (11) y Núcleo (10) suman 33 de los 55 requerimientos. La consistencia entre el formato físico y el digital es, según Pezoa Riquelme (2020), uno de los aspectos que distinguen a EDDIE como propuesta para investigar el efecto de la lectura aumentada en la comprensión lectora. Los módulos de plugins cuentan con tres requerimientos cada uno porque, como indica la Tabla 3.1, establecen contratos de interfaz (PluginBase) y no funciones específicas de cada dispositivo.

3.2.3 Requerimientos no funcionales

Los requerimientos no funcionales establecen las cualidades que debe cumplir el sistema independientemente de las funciones que ofrezca. La Tabla 3.2 especifica los ocho requerimientos que rigen el diseño y la operación de la nueva arquitectura en Python, derivados de las restricciones de calidad, rendimiento y mantenibilidad establecidas para el proyecto. Los requerimientos RNF-01 a RNF-05 guardan correspondencia con los requerimientos no funcionales de Multimodal Observer (Figueroa Contreras, 2025).

Tabla 3.2: Requerimientos no funcionales.

ID	Requerimiento no funcional	Criterio de aceptación / Fuente
RNF-01	Multiplataforma	El sistema debe ejecutarse en Windows 10/11, Linux (Ubuntu) y macOS utilizando un runtime estándar de Python 3.14 y paquetes multiplataforma de PyPI. (Adaptado de Figueroa Contreras, 2025)
RNF-02	Modularidad y extensibilidad estricta	El sistema debe permitir integrar nuevas funcionalidades mediante plugins desacoplados que hereden de PluginBase, sin requerir modificaciones en el código fuente del núcleo.
RNF-03	Internacionalización (i18n)	La interfaz y los módulos deben soportar el cambio dinámico de idioma entre español e inglés mediante diccionarios JSON traducibles.
RNF-04	Documentación y mantenibilidad	El código fuente debe contar con una cobertura del 100 % de comentarios docstring bajo el estándar PEP 257, junto con las convenciones de estilo de PEP 8, e incluir guías claras para el desarrollo de nuevos plugins.
RNF-05	Carga dinámica en tiempo de ejecución	El módulo PluginLoader debe poder instanciar, cargar y descargar plugins en tiempo de ejecución, sin recompilar la aplicación ni usar dominios de aplicación inestables (AppDomains).
RNF-06	Consistencia espacio-temporal físico-digital	La conversión de coordenadas entre la superficie del papel impreso y los documentos PDF digitales debe ser homotética y exacta, utilizando puntos tipográficos (pt) y QuadPoints de ocho vértices sin desalineación.
RNF-07	Eficiencia y gestión de memoria	El procesamiento de imágenes con OpenCV debe liberar explícitamente los buffers de memoria (arreglos NumPy), evitando congelamientos o degradación del sistema por uso prolongado.
RNF-08	Resiliencia y tolerancia a fallos	La falla o excepción en la ejecución de un plugin no debe provocar la caída global del sistema; el orquestador debe aislar el error, registrar el evento en el log y continuar la ejecución de forma defensiva.

Fuente: Elaboración propia, 2026.

Varios de los requerimientos no funcionales se relacionan con errores documentados en el sistema legado por Ibaceta Jaña (2023). Los requerimientos RNF-02 y RNF-05 se vinculan con la carga de plugins como archivos .dll que debían compilarse y ubicarse manualmente en carpetas específicas, lo que produjo errores como la ausencia de la carpeta PluginsConsistencia (EC15), de los plugins de rastreo ocular (ERO2) y del plugin consistencyLibraryFiguresDP.dll (EC24). El RNF-08 se relaciona con las fallas provocadas por variables con valor nulo (ERO3 a ERO6 y EC17 a EC19). El RNF-06 se relaciona con el error de destacado de texto que no coincidía en posición ni largo entre el formato físico y el digital (EC6), y el RNF-07 con el error de reconocimiento de texto que dejaba de funcionar tras un uso prolongado (EFD3). Finalmente, el RNF-03 se relaciona con los mensajes de error en un idioma distinto al del resto de la aplicación (EO6), y el RNF-04 con la falta de comentarios en el código, de nombres representativos y de manuales de instalación y de uso (EO11, EO12, EE6 y EE7).

Estos ocho requerimientos constituyen además el criterio de verificación de la fase siguiente: cada prototipo se evalúa explícitamente frente a aquellos que le son aplicables, como se aprecia en las fichas de validación de la sección de prototipado.

3.3 FACTIBILIDAD TÉCNICA

3.4 PROTOTIPADO

Esta sección presenta la fase de prototipado del proyecto, cuyo propósito es reducir la incertidumbre técnica antes de abordar el diseño e implementación de la solución definitiva. Dado que el sistema EDDIE-2023 fue desarrollado originalmente en C# y depende de bibliotecas específicas de Windows, resulta necesario comprobar, de manera temprana y con bajo costo, que sus funciones esenciales pueden reproducirse en Python sin pérdida de capacidades. Con este fin se construyeron prototipos de carácter desechable (throwaway), es decir, piezas de software acotadas cuyo valor reside en los hallazgos que entregan y no en el código que producen.

Durante las primeras semanas del proyecto, cada prototipo se desarrolló en un entorno Python aislado de las dependencias de hardware y de interfaz gráfica del sistema legado. Este aislamiento permitió evaluar la factibilidad técnica de cada función en función de tres criterios: (i) la equivalencia funcional respecto de la implementación en C#, (ii) la compatibilidad de las bibliotecas disponibles en el ecosistema Python con los requerimientos del sistema, y (iii) el cumplimiento de los Requisitos Funcionales (RF) de cada módulo (Tabla 3.1) y de los Requisitos No Funcionales de calidad y rendimiento (RNF-01 a RNF-08, Tabla 3.2).

Como paso previo, se realizó un levantamiento exhaustivo de las funciones que componen el código original. Este inventario permitió identificar cuáles resultan relevantes para el alcance del proyecto y cuáles quedan fuera de él, ya sea por depender de hardware no disponible o por tratarse de servicios externos que pueden integrarse sin un prototipo previo. Las funciones se agruparon en siete subsistemas según su responsabilidad dentro de la arquitectura y se presentan en las Tablas 3.3 a 3.9. Cada tabla indica el identificador de la función, su equivalente en el código legado, una descripción, el hardware que requiere y la decisión de prototiparla o no, junto con la tecnología Python propuesta como reemplazo.

Tabla 3.3: Funciones del subsistema de procesamiento de imágenes y visión computacional.

ID	Función EDDIE-2023	Archivos C# Legacy	Descripción de la Función	Hardware Requerido	¿Prototipar?
F1	Enumeración e Inicialización de Cámaras	Módulo Procesamiento Imágenes / CameraActivity.cs	Detecta, enumera y configura las cámaras conectadas mediante DirectShow.	Cámara 1 (Texto) y/o Cámara 2 (Gestos)	SÍ (OpenCV cv2.VideoCapture)
F2	Preprocesamiento y Binarización de Texto	Módulo Procesamiento Imágenes / ColorRecognition.cs	Recorta la imagen del papel, convierte BGR a escala de grises, aplica filtro Gaussiano y binarización Otsu para aislar el texto impreso.	Cámara 1 (Texto / Documentos)	SÍ (OpenCV cv2.threshold)
F3	Reconocimiento Óptico de Caracteres (OCR)	Módulo Procesamiento Imágenes / OCRProcess.cs	Convierte la imagen recortada y preprocesada en texto legible en español e inglés usando el motor Tesseract.	Ninguno (solo software / Tesseract binary)	SÍ (PyTesseract / EasyOCR)
F4	Detección Automática de Páginas del Libro	Augmented ReadingApp / PageDetectionSettings.cs	Detecta los bordes del libro impreso para ajustar el área de proyección y compensar la deformación de la página.	Cámara 1 (Texto) + Proyector	SÍ (OpenCV findContours)

Fuente: Elaboración propia.

El subsistema de procesamiento de imágenes constituye la base perceptiva de EDDIE, pues es el encargado de transformar lo que la cámara observa sobre el escritorio en información que el resto del sistema puede interpretar. Sus cuatro funciones forman una cadena de dependencias: la correcta inicialización de las cámaras (F1) habilita la captura de imágenes, el preprocesamiento y la binarización (F2) mejoran su calidad, el OCR (F3) extrae el texto y la detección de páginas (F4) delimita el área de trabajo. Dado que un fallo en cualquiera de estos eslabones degrada todas las funcionalidades posteriores, se decidió prototipar las cuatro funciones. Además, todas cuentan con alternativas maduras en Python, principalmente OpenCV y Tesseract, lo que hace esperable una migración sin pérdida de funcionalidad.

Tabla 3.4: Funciones del subsistema de reconocimiento gestual e interacción sin contacto.

ID	Función EDDIE-2023	Archivos C# Legacy	Descripción de la Función	Hardware Requerido	¿Prototipar?
F5	Seguimiento de Puntero / Lápiz de Color (ColorPen)	ColorPen Recognition / ColorPen.cs	Segmenta un marcador o lápiz de color (azul/rojo) en el espacio HSV/RGB para usarlo como puntero interactivo sobre el papel.	Cámara 2 (Gestos / Escritorio)	SÍ (OpenCV inRange HSV)
F6	Segmentación de Mano y Apuntado con Dedo (HandSkin)	Hand Skin Recognition / YCrCbSkin Detector.cs	Aísla la piel humana en espacio YCrCb, calcula contornos y defectos de convexidad para detectar la punta del dedo índice y contar dedos levantados.	Cámara 2 (Gestos / Escritorio)	SÍ (OpenCV YCrCb + convexity Defects)
F7	Mapeo de Coordenadas de Gestos a Pantalla	Módulo Reconocimiento Gestual / GestureRecognitionActivity.cs	Transforma las coordenadas (X,Y) del sensor de cámara a la resolución del proyector y simula clics mediante Win32 P/Invoke.	Cámara 2 + Proyector	SÍ (Calibración cv2.find Homography)
F8	Reconocimiento Gestual 3D (LeapMotion)	Leap Motion Recognition / Leap Motion Plugin.cs	Captura gestos de las manos en 3D mediante el sensor nativo Leap Motion.	Controller Leap Motion 3D	No (Fuera de alcance MVP laboratorio)
F9	Fallback de Entrada por Mouse (Mouse Recognition)	Mouse Recognition / Mouse Plugin.cs	Permite usar el puntero del ratón como dispositivo de interacción secundario.	Mouse / Teclado	SÍ (Mouse fallback integrado)

Fuente: Elaboración propia.

Este subsistema permite al lector interactuar con el documento sin contacto con dispositivos tradicionales, ya sea mediante un lápiz de color (F5) o mediante su propia mano (F6). La función F7 completa el flujo al traducir las coordenadas de la cámara al espacio del proyector, de modo que el gesto detectado se convierta en una acción sobre la interfaz proyectada. Se seleccionaron para prototipado cuatro de las cinco funciones. La excepción es F8 (Leap Motion), descartada por depender de un sensor 3D propietario que escapa al alcance del MVP de laboratorio. En su lugar, la función F9 incorpora el ratón como mecanismo de respaldo, garantizando que el sistema pueda operarse incluso cuando el reconocimiento por cámara no esté disponible.

Tabla 3.5: Funciones del subsistema de rastreo ocular y atención visual (Eye Tracking).

ID	Función EDDIE-2023	Archivos C# Legacy	Descripción de la Función	Hardware Requerido	¿Prototipar?
F10	Conexión a Rastreador Ocular por Socket / WebSocket	Plugin GazeCloud (WatsonWsServer) y Plugin EyeTribe	Abre un servidor socket local en puerto 3000 (GazeCloud) o 6555 (EyeTribe TCP) para recibir coordenadas de mirada en tiempo real (30-60 Hz).	Eye Tracker (GazeCloud / EyeTribe)	SÍ (Sockets asyncio / websockets)
F11	Carga y Aislamiento de Plugins Eyetracker	Módulo Rastreo Ocular / IntermediateClass.cs	Instancia DLLs de rastreo ocular en dominios de aplicación C# (AppDomain / MarshalByRefObject).	Ninguno (Arquitectura)	SÍ (Reemplazar por importlib de Python)
F12	Detección de Fijación Atencional (Dwell Time)	Módulo Rastreo Ocular / MouseControl.cs	Mide si la mirada permanece fija en un radio de píxeles durante >400 ms para gatillar la selección o despliegue de ayuda.	Eye Tracker / Mouse	SÍ (Filtro temporal Dwell)
F13	Renderizado de Retículo de Mirada proyectado	Módulo Rastreo Ocular / ReticleDrawing.cs	Dibuja una mira o retículo visual en el proyector exactamente sobre el punto donde el lector fija su mirada.	Eye Tracker + Proyector Superior	SÍ (Overlay visual PyGame / OpenCV)

Fuente: Elaboración propia.

El rastreo ocular es el subsistema que distingue a EDDIE como herramienta de investigación, ya que genera los datos de atención visual con los que se estudia la comprensión lectora. Las cuatro funciones se prototiparon, pues cubren el ciclo completo: recepción de coordenadas desde el dispositivo (F10), carga dinámica de los controladores específicos de cada rastreador (F11), interpretación de la mirada como acción mediante el tiempo de fijación (F12) y retroalimentación visual al lector (F13). Una consideración particular es la función F11, que no depende de hardware sino de la arquitectura de plugins; su migración a importlib es clave para conservar la extensibilidad del sistema, es decir, la capacidad de integrar nuevos dispositivos sin modificar el núcleo.

Tabla 3.6: Funciones del subsistema de consistencia físico-digital en PDFs.

ID	Función EDDIE-2023	Archivos C# Legacy	Descripción de la Función	Hardware Requerido	¿Prototipar?
F14	Búsqueda y Mapeo de Coordenadas QuadPoints en PDF	Módulo Consistencia Datos / Digital-DocSync.cs	Busca palabras dentro del PDF digital usando iTextSharp y recupera las coordenadas exactas (QuadPoints / bounding boxes) de cada línea.	Ninguno (Archivo PDF)	SÍ (PyMuPDF fitz.search _ for)
F15	Sincronización de Destacado (Highlight)	Módulo Consistencia Datos / Digital-DocSync.cs	Estampa franjas de subrayado amarillo translúcido en el archivo PDF digital cuando el usuario subraya sobre el papel físico.	Cámara 1 + Proyector + PDF	SÍ (PyMuPDF add_ highlight _ annot)
F16	Sincronización de Notas Adhesivas (Post-It / Comentarios)	Consistency Library CommentsDP / Com-mentsPD.cs	Sincroniza notas escritas en el papel hacia el PDF digital como anotaciones de texto, o proyecta post-its digitales sobre el libro físico.	Cámara 1 + Proyector + PDF	SÍ (PyMuPDF add_text _ annot)
F17	Sincronización de Figuras Geométricas	Consistency Library FiguresDP / FiguresPD.cs	Extrae o dibuja círculos y rectángulos sobre el PDF y los proyecta sobre el libro de texto impreso.	Proyector + Archivo PDF	SÍ (PyMuPDF add_rect _ annot)
F18	Sincronización de Marcapáginas	Consistency Library Page MarkerDP / PageMark-erPD.cs	Sincroniza las pestañas de separación y marcadores de página entre la versión impresa y la versión PDF.	Archivo PDF	No (PyMuPDF TOC / Bookmarks)

Fuente: Elaboración propia.

El módulo de consistencia mantiene la equivalencia entre el documento impreso y su versión digital, de manera que las marcas realizadas en uno se reflejen en el otro. Su funcionamiento descansa en la capacidad de localizar texto dentro del PDF y obtener sus coordenadas exactas (F14); sobre esa base se construyen la sincronización de destacados (F15), de notas (F16) y de figuras (F17). Estas cuatro funciones se prototiparon aprovechando que PyMuPDF ofrece, en una única biblioteca, tanto la búsqueda con coordenadas como la creación de anotaciones, lo que simplifica la migración respecto de iTextSharp. La función F18 (marcapáginas) se excluyó del prototipado porque su equivalente en Python, la gestión de marcadores y tabla de contenidos de PyMuPDF, es una operación directa que no presenta riesgo técnico relevante.

Tabla 3.7: Funciones del subsistema de búsqueda web de conocimiento y APIs REST.

ID	Función EDDIE-2023	Archivos C# Legacy	Descripción de la Función	Hardware Requerido	¿Prototipar?
F19	Búsqueda Enciclopédica (Wikipedia REST API)	Módulo Búsqueda Web / Api-Wikipedia.cs	Consulta la API REST de Wikipedia en español para traer resúmenes de términos seleccionados.	Conexión a Internet	No (urllib + Regex Sanitizer)
F20	Búsqueda de Definiciones Conceptuales	Módulo Búsqueda Web / ApiDefini-ciones-Google.cs	Consulta diccionarios JSON en línea para mostrar significados de palabras (Propuesto migrar a Ollama LLM local).	Internet / Servidor Ollama	No (Conector REST Ollama local-host:11434)
F21	Traducción de Texto (Bing / Azure Translator)	Módulo Búsqueda Web / ApiTraduc-cionBing.cs	Traduce fragmentos de texto seleccionados de inglés a español.	Conexión a Internet	No (API REST Azure / DeepL / Local)
F22	Reconocimiento de Imágenes de Libros (Cloud Vision)	Módulo Búsqueda Web / ApiBusquedaIm-agenCloudVi-sion.cs	Codifica la foto de una ilustración o esquema en Base64 y consulta Google Cloud Vision API.	Cámara 1 + Internet	No (For-mateador Payload Base64)
F23	Búsqueda y Reproducción de Videos Explicativos	Módulo Búsqueda Web / ApiBuscarY-outube.cs	Construye URLs de búsqueda en YouTube y despliega videos relevantes para complementar la lectura.	Internet + Altavoces	No (YouTube Query Builder)

Fuente: Elaboración propia.

A diferencia de los subsistemas anteriores, ninguna de las cinco funciones de búsqueda web fue seleccionada para prototipado. La razón es que todas consisten en consultas a servicios externos mediante APIs REST, cuya integración en Python es una tarea estándar y de bajo riesgo técnico. Sin embargo, este subsistema presenta un riesgo distinto, de tipo operacional: varios de los servicios utilizados por el sistema legado ya no están disponibles en las condiciones originales, por ejemplo credenciales de estudiantes caducadas o versiones de prueba expiradas. Por ello, para estas funciones se propone sustituir los proveedores por alternativas vigentes, como un modelo de lenguaje local mediante Ollama en el caso de las definiciones (F20), y su integración se abordará directamente en la implementación final.

Tabla 3.8: Funciones del subsistema de voz, audio e intenciones.

ID	Función EDDIE-2023	Archivos C# Legacy	Descripción de la Función	Hardware Requerido	¿Prototipar?
F24	Reconocimiento de Comandos por Voz (STT)	Augmented ReadingApp / Projection-ScreenActiv-ity2.cs	Escucha la voz del lector para ejecutar intenciones (“buscar enciclopedia”, “traducir”) usando SAPI5 (Propuesto migrar a Whisper).	Micrófono de audio	SÍ (OpenAI Whisper STT)
F25	Síntesis de Voz (Text-to-Speech / TTS)	Augmented ReadingApp / Projection-ScreenActiv-ity2.cs	Lee en voz alta las definiciones, traducciones o resúmenes al usuario usando el motor nativo de Windows.	Altavoces / Salida de audio	SÍ (pyttsx3 / SAPI5 Python)

Fuente: Elaboración propia.

El subsistema de voz permite una interacción manos libres, coherente con el carácter no obstructivo de EDDIE: el lector emite una orden hablada (F24) y recibe la respuesta de forma auditiva (F25). Ambas funciones se prototiparon porque su migración implica un cambio tecnológico significativo, desde el motor SAPI5 propio de Windows hacia Whisper en el reconocimiento de voz, lo que exige verificar la calidad del reconocimiento y la latencia obtenida. Por su parte, la síntesis de voz con pyttsx3 se prototipó para confirmar que el motor nativo del sistema operativo sigue siendo accesible desde Python y que no introduce nuevas dependencias.

Tabla 3.9: Funciones del subsistema de presentación GUI, proyección y logs.

ID	Función EDDIE-2023	Archivos C# Legacy	Descripción de la Función	Hardware Requerido	¿Prototipar?
F26	Navegador Embebido (CefSharp / CEF)	Augmented ReadingApp / Projection-ScreenActiv-ity2.cs	Renderiza páginas web, videos de YouTube y resultados HTML sobre la proyección en el escritorio.	Proyector Superior	No (PyQt / WebEngine / Browser)
F27	Herramienta de Subrayado Digital proyectado (HighlightTool)	Módulo Visualización Datos / High-lightTool.cs	Dibuja una banda rectangular amarilla transparente interactiva en pantalla sobre la proyección.	Proyector Superior	SÍ (Canvas transparente OpenCV / PyGame)
F28	Registro Asíncrono de Eventos y Mirada (ModuloLog)	ModuloLog / StandardLog-ging.cs	Guarda marcas de tiempo, eventos de interacción y coordenadas de la mirada usando Serilog asíncrono en CSV/JSON.	Ninguno (Disco / Almace-namiento)	No (Python logging / structlog)

Fuente: Elaboración propia.

Este subsistema reúne los elementos de presentación y registro que dan soporte al resto de las funcionalidades. Solo el subrayado proyectado (F27) se seleccionó para prototipado, por ser el único con requisitos gráficos particulares, ya que debe superponer una banda translúcida e interactiva sobre el documento físico. El navegador embebido (F26) y el registro de eventos

(F28) se descartaron porque cuentan con equivalentes directos y ampliamente documentados en Python, y su integración no plantea incertidumbre técnica. El registro de eventos reviste especial importancia para la investigación, dado que almacena las coordenadas de la mirada y los eventos de interacción que serán analizados posteriormente.

3.4.1 Síntesis de la selección de funciones

Del total de 28 funciones identificadas, 19 fueron seleccionadas para prototipado y 9 quedaron excluidas. Los criterios de exclusión fueron tres: dependencia de hardware fuera del alcance del MVP (F8), equivalencia directa en Python que no representa riesgo técnico (F18, F19, F21 a F23, F26 y F28) y migración condicionada a la sustitución del proveedor externo (F20). Esta selección concentra el esfuerzo de validación en las funciones que combinan mayor incertidumbre tecnológica con mayor impacto sobre el objetivo del sistema, que es habilitar la investigación en lectura aumentada. Los prototipos se desarrollaron de manera incremental, uno por función, y sus resultados se documentan mediante fichas de validación con una estructura común: objetivo, librerías y hardware, contrato o interfaz, requisitos asociados, y resultados y hallazgos.

3.4.2 Prototipo F1: Enumeración e Inicialización de Cámaras

El primer prototipo corresponde a la función de enumeración e inicialización de cámaras (F1). En el sistema legado, esta función detecta y configura las cámaras mediante DirectShow; el prototipo busca reproducir ese comportamiento en Python con OpenCV y obtener los nombres reales de los dispositivos mediante consultas WMI invocadas desde PowerShell, de modo de asignar a cada cámara su rol (captura de texto o de gestos). La Tabla 3.10 resume las características y resultados de la validación.

Tabla 3.10: Ficha de validación del prototipo: Enumeración e Inicialización de Cámaras.

Objetivo	Detectar, enumerar y configurar múltiples cámaras USB/DirectShow en Python (Cámara de Texto y Cámara de Gestos).
Librerías / Hardware	OpenCV (cv2.VideoCapture), NumPy, PowerShell (Get-CimInstance Win32_PnPEntity) / Cámaras USB 1 y 2.
Contrato / Interfaz	IImageProcessor.
Requisitos Asociados	• **RF (Captura de Datos y Sensores):** Iniciar, pausar, reanudar y detener capturas síncronas de video de cámaras. • **RNF-01 (Multiplataforma):** Ejecución nativa mediante binding multiplataforma de OpenCV en Python 3.14. • **RNF-07 (Eficiencia y Gestión de Memoria):** Liberación explícita de buffers de video y arreglos NumPy al liberar la cámara (cap.release()).
Resultados y Hallazgos	• **Tasa de cuadros / Latencia:** 29.98 ± 0.42 FPS estables a resolución 640×480 px con una latencia media de captura de 14.2 ± 1.1 ms por cuadro. • **Manejo de dispositivos:** Asignación dinámica de índices 0 (Webcam integrada) e 1/2 (Cámara cenital DroidCam/USB), obteniendo los nombres reales del dispositivo mediante invocación WMI/PowerShell en Windows 10/11. • **Estado:** Validado y completado. Lógica integrada en la capa de abstracción de hardware en Python.

Fuente: Elaboración propia.

Protocolo de validación. Se capturó una secuencia continua de 1 800 cuadros (60 s a 30 FPS nominales) de forma simultánea en dos dispositivos: la webcam integrada y la cámara cenital DroidCam conectada por USB. La tasa efectiva de cuadros se calculó como

$$\text{FPS} = \frac{N}{\sum_{i=1}^{N} \Delta t_i},$$

y la latencia se definió como el intervalo entre la invocación de cap.read() y la disponibilidad de la matriz BGR en la memoria principal. Los tiempos se midieron con time.perf_counter() y la memoria con tracemalloc y psutil.

Los resultados del prototipo F1 indican que la gestión de cámaras puede migrarse a Python con OpenCV. La captura se mantuvo en 29.98 ± 0.42 FPS con una latencia media de 14.2 ± 1.1 ms por cuadro, inferior a los 33.3 ms que corresponden a un cuadro a esa tasa, utilizando el backend Microsoft Media Foundation con respaldo automático a cv2.CAP_ANY. La combinación de OpenCV con consultas WMI permitió asignar de forma dinámica los índices de cámara y obtener los nombres reales de los dispositivos en Windows 10/11 sin colisiones de identificador. Tras liberar el dispositivo con cap.release(), la memoria residual fue de 0.0 MB, lo que no evidenció fugas y responde al RNF-07; esto es relevante porque Ibaceta Jaña (2023) documentó que una cámara en uso por otra aplicación impide el correcto funcionamiento de EDDIE. El prototipo queda validado y su lógica se integra en la capa de abstracción de hardware en Python.

3.4.3 Prototipo F2: Preprocesamiento y Binarización de Texto

El prototipo F2 corresponde a la función de pre-procesamiento y binarización de texto, del subsistema de procesamiento de imágenes. En el sistema legado (ColorRecognition.cs), esta función recorta la imagen del papel, la convierte de BGR a escala de grises, aplica un filtro Gaussiano y una binarización de Otsu para aislar el texto impreso (Tabla 3.3). El prototipo reproduce esa secuencia con OpenCV como etapa previa al reconocimiento óptico de caracteres (F3), y mide la reducción de destellos y sombras y el tiempo de procesamiento por cuadro. La Tabla 3.11 resume la validación.

Tabla 3.11: Ficha de validación del prototipo: Preprocesamiento y Binarización de Texto.

Objetivo	Recortar el área del documento, convertir a escala de grises, aplicar filtro Gaussiano y binarización de Otsu para aislar texto impreso.
Librerías / Hardware	OpenCV (cv2.threshold, cv2.GaussianBlur, cv2.cvtColor) / Cámara de Documentos.
Contrato / Interfaz	IImageProcessor.
Requisitos Asociados	• **RF (Plugins de Captura y Hardware - HAL):** Extender procesadores de imagen mediante contratos de interfaz definidos (PluginBase). • **RNF-07 (Eficiencia y Gestión de Memoria):** Procesamiento de matrices OpenCV en memoria con reutilización/destrucción explícita de buffers para evitar fugas de memoria.
Resultados y Hallazgos	• **Calidad de imagen:** La binarización de Otsu eliminó el 96.4% de los destellos y sombras provocados por la luz directa del proyector sobre el papel blanco impreso. • **Tiempo de procesamiento:** 3.12 ± 0.28 ms por cuadro (640×480 px), operando a más de 320 FPS en CPU. • **Estado:** Validado y completado. Base para el pipeline de preprocesamiento del motor OCR.

Fuente: Elaboración propia.

Protocolo de validación. Se utilizó un conjunto de 120 imágenes de documentos impresos bajo las condiciones adversas que genera el proyector cenital (lámpara DLP de 2 000 lúmenes sobre papel bond de 75 g/m^2): 40 con sobreexposición especular ($V > 240$ en HSV), 40 con sombras parciales de manos u objetos ($V < 60$) y 40 con gradientes de iluminación transversales (300 a 850 lux). Para cada imagen se etiquetó manualmente, a nivel de píxel, una máscara de verdad terreno que distingue texto y fondo, y la eficacia se definió como

$$\text{Eficacia} = \left(1 - \frac{FP_{\text{ruido}} + FN_{\text{corte}}}{P_{\text{afectados}}}\right) \times 100\%,$$

donde $P_{\text{afectados}}$ es el total de píxeles en zonas de deslumbramiento o penumbra, FP_{ruido} los píxeles de destello que se confunden con tinta y FN_{corte} los píxeles de caracteres perdidos o fragmentados por la sombra. El tiempo se midió con cv2.getTickCount() sobre 1 000 iteraciones continuas en

CPU.

El prototipo F2 muestra que la secuencia de pre-procesamiento del sistema legado puede reproducirse con OpenCV. La binarización de Otsu alcanzó una eficacia del 96.4% en la eliminación de destellos y sombras producidos por la luz del proyector sobre el papel, evaluada sobre las 120 imágenes con máscaras de verdad terreno; esta condición es propia de EDDIE, cuya proyección cubre el escritorio y el documento (requisito RFCO1, según Ibaceta Jaña (2023)). El tiempo de 3.12 ± 0.28 ms por cuadro, que permitiría más de 320 FPS frente a los 30 FPS del sensor, corresponde únicamente a esta etapa, por lo que el costo total del flujo dependerá también del reconocimiento de texto (F3). Con ello, el prototipo queda validado como base del pipeline de preprocesamiento del motor OCR.

3.4.4 Prototipo F3: Reconocimiento Óptico de Caracteres

El prototipo F3 corresponde al reconocimiento óptico de caracteres. En el sistema legado (OCRProcess.cs), esta función convierte la imagen recortada y preprocesada en texto legible en español e inglés mediante el motor Tesseract (Tabla 3.3), motor que ya utilizaban los desarrollos de Carrasco Madriaga (2017) y Contreras Velásquez (2019) según Ibaceta Jaña (2023). El prototipo evalúa el motor mediante pytesseract sobre imágenes preprocesadas, con métricas de tasa de error por palabra (WER) y por carácter (CER), y su tiempo de inferencia. La Tabla 3.12 resume la validación.

Tabla 3.12: Ficha de validación del prototipo: Reconocimiento Óptico de Caracteres.

Objetivo	Convertir imágenes de texto preprocesadas en cadenas de caracteres legibles en español e inglés mediante Tesseract.
Librerías / Hardware	pytesseract (Tesseract OCR v5.x Engine) / Cámara de Documentos.
Contrato / Interfaz	IImageProcessor.
Requisitos Asociados	• **RF (Consistencia Físico-Digital - F16):** Detección e inyección de anotaciones y reconstrucción de texto desde el papel impreso. • **RNF-03 (Internacionalización - i18n):** Soporte multilingüe en extracción de texto (español/inglés) con diccionarios de idioma (spa+eng). • **RNF-08 (Resiliencia y Tolerancia a Fallos):** Captura defensiva ante errores de reconocimiento de caracteres en imágenes con baja resolución o sombra.
Resultados y Hallazgos	• **Precisión (CER/WER):** Tasa de error por palabra (WER) de 4.24% y tasa de error por carácter (CER) de 1.12% sobre texto impreso en tipografía Times New Roman / Arial de 10, 11 y 12 pt. • **Tiempo de inferencia:** 175.4 ± 18.2 ms promedio por bloque de párrafo recortado. • **Estado:** Validado y completado. Módulo migrado como servicio nativo en Python.

Fuente: Elaboración propia.

Protocolo de validación. El corpus estuvo compuesto por 25 páginas impresas en papel bond de 75 g/m^2, con textos académicos de ingeniería y fragmentos del informe de seminario de EDDIE, que totalizaron 8 450 palabras y 52 320 caracteres (con tildes, signos de puntuación, mayúsculas y números). Se imprimieron a 300 DPI en Times New Roman y Arial, en tamaños de 10, 11 y 12 pt. Las tasas de error se calcularon con la distancia de Levenshtein respecto de la verdad terreno:

$$\text{CER} = \frac{S_c + D_c + I_c}{N_c} \times 100\%, \qquad \text{WER} = \frac{S_w + D_w + I_w}{N_w} \times 100\%,$$

donde S, D e I son las sustituciones, supresiones e inserciones, y $N_c = 52\,320$ y $N_w = 8\,450$ son los totales de referencia. La transcripción se obtuvo con Tesseract v5.4.0 (opciones –oem 3 –psm 6 -l spa+eng) y se comparó con el módulo Levenshtein de Python.

El prototipo F3 obtuvo una tasa de error por carácter (CER) de 1.12% (586 errores en 52 320 caracteres, concentrados principalmente en tildes y comillas curvas) y una tasa de error por palabra (WER) de 4.24% (358 palabras erróneas de 8 450), sobre texto impreso en Times New Roman y Arial de 10, 11 y 12 pt, con un tiempo de inferencia de 175.4 ± 18.2 ms por bloque de párrafo recortado de 400×180 px. El uso del diccionario spa+eng responde al soporte de español e inglés exigido por el RNF-03. Estas métricas no son directamente comparables con la validez por palabra y posición medida por Ibaceta Jaña (2023), que varió entre 10% y 100% según la ubicación de la palabra respecto de la cámara. Cabe notar que la ficha no reporta el efecto de la posición del texto, factor que ese trabajo identificó como determinante, por lo que esa condición no está cubierta por esta validación. El prototipo queda validado como servicio nativo en Python.

3.4.5 Prototipo F4: Detección Automática de Páginas

El prototipo F4 corresponde a la detección automática de páginas. En el sistema legado (PageDetectionSettings.cs), esta función detecta los bordes del libro impreso para ajustar el área de proyección y compensar la deformación de la página (Tabla 3.3). El prototipo busca reproducir esa detección con OpenCV, hallando el contorno de la página y corrigiendo su perspectiva. La Tabla 3.13 resume la validación.

Tabla 3.13: Ficha de validación del prototipo: Detección Automática de Páginas.

Objetivo	Detectar los bordes del libro impreso para ajustar el área de proyección y corregir la perspectiva de la página.
Librerías / Hardware	OpenCV (cv2.findContours, cv2.approxPolyDP, cv2.getPerspectiveTransform) / Cámara 1 + Proyector.
Contrato / Interfaz	IImageProcessor / IProjectionService.
Requisitos Asociados	• **RF (Plugins de Visualización y Proyección):** Desplegar overlays de proyección alineados sobre el papel físico. • **RNF-06 (Consistencia Espacio-Temporal Físico-Digital):** Garantizar la transformación homotética exacta del contorno de la página física.
Resultados y Hallazgos	• **Ajuste geométrico:** Estabilidad del cuadrilátero con un error cuadrático medio (RMSE) de 1.38 pt (≈ 0.48 mm) en las 4 esquinas del libro, con una tasa de éxito del 98.75% (79 de 80 ensayos), bajo iluminación ambiente de 300 a 750 lux. • **Estado:** Validado y completado. Algoritmo reestructurado para el módulo de calibración inicial de proyección.

Fuente: Elaboración propia.

Protocolo de validación. Se utilizó un banco de 80 cuadros de libros físicos en formatos Carta (215.9×279.4 mm) y A4 (210×297 mm), sobre superficies de contraste alto y medio, con orientaciones entre $-15°$ y $+15°$ y cuatro niveles de iluminancia ambiental entre 300 y 750 lux. Las cuatro esquinas estimadas con cv2.approxPolyDP (tolerancia $\epsilon = 0.02\times$ perímetro) se compararon con las coordenadas $(\hat{x}, \hat{y})$ marcadas manualmente a nivel sub-píxel mediante

$$\text{RMSE} = \sqrt{\frac{1}{4N}\sum_{i=1}^{N}\sum_{k=1}^{4}((x_{k,i}-\hat{x}_{k,i})^2+(y_{k,i}-\hat{y}_{k,i})^2)}.$$

El prototipo F4 logró un RMSE de 1.38 pt (≈ 0.48 mm) en las cuatro esquinas y una tasa de éxito del 98.75% en la identificación del cuadrilátero (79 de 80 ensayos), con libros Carta y A4 inclinados entre $-15°$ y $+15°$ y cuatro niveles de iluminación de 300 a 750 lux. El resultado se acota a ese rango de condiciones: Ibaceta Jaña (2023) señaló la iluminación y la posición de la cámara como causas posibles del desfase del destacado entre el formato físico y el digital (EC6), por lo que la sensibilidad a la luz debe comprobarse en la integración. El algoritmo se reestructuró como módulo de calibración inicial de la proyección y responde al RNF-06, relativo a la transformación homotética del contorno de la página.

3.4.6 Prototipo F5: Seguimiento de Puntero de Color

El prototipo F5 corresponde al seguimiento de un lápiz o marcador de color como puntero, del subsistema de reconocimiento gestual. En el sistema legado (ColorPen.cs), esta función segmenta un marcador azul o rojo en el espacio HSV/RGB para usarlo como puntero

sobre el papel (Tabla 3.4); en el desarrollo de Contreras Velásquez (2019), la posición de la punta de color se utiliza para controlar el puntero, según Ibaceta Jaña (2023). El prototipo segmenta el color en el espacio HSV y lo implementa como plugin gestual. La Tabla 3.14 resume la validación.

Tabla 3.14: Ficha de validación del prototipo: Seguimiento de Puntero de Color.

Objetivo	Segmentar un lápiz o marcador de color (rojo/azul) en el espacio HSV para usarlo como puntero físico.
Librerías / Hardware	OpenCV (cv2.inRange, HSV, cv2.morphologyEx) / Cámara de Escritorio.
Contrato / Interfaz	IGesturePlugin.
Requisitos Asociados	• **RF (Plugins de Captura y Hardware - HAL):** Extender sensores gestuales mediante contratos estrictos herederos de PluginBase. • **RNF-02 (Modularidad y Extensibilidad Estricta):** Implementación desacoplada como plugin gestual sin modificar el orquestador central. • **RNF-07 (Eficiencia y Gestión de Memoria):** Segmentación por color en tiempo real optimizada sobre arreglos de NumPy.
Resultados y Hallazgos	• **Sensibilidad a la luz:** La conversión a espacio HSV (H: 0–10 y 170–180 para rojo; S: 120–255; V: 100–255), combinada con una clausura morfológica, eliminó el 92% de los falsos reflejos generados por la proyección digital sobre la superficie del lápiz. • **Estado:** Validado y completado. Plugin de entrada gestual básica incorporado.

Fuente: Elaboración propia.

Protocolo de validación. Se registró una secuencia continua de 1 200 cuadros (40 s a 30 FPS) con un marcador Sharpie rojo de 12 mm de diámetro sobre papel impreso, mientras el proyector mostraba diapositivas dinámicas de alto contraste con alternancia rápida de rojo, blanco, verde y cian. Se contabilizaron como detecciones erróneas los centroides espurios o saltos bruscos de más de 30 px entre cuadros contiguos, generados por la luz proyectada reflejada en el plástico del marcador. Con la implementación legada en C# basada en el espacio RGB se registraron 148 detecciones espurias por minuto; con la segmentación HSV dual ($H_1 \in [0, 10]$, $H_2 \in [170, 180]$, $S \in [120, 255]$, $V \in [100, 255]$) y una clausura morfológica con elemento estructurante elíptico de 9×9, se registraron 12 en los 1 200 cuadros.

El prototipo F5 redujo las detecciones espurias de 148 por minuto en la implementación legada basada en RGB a 12 en la secuencia evaluada, lo que se reporta como una atenuación de falsos reflejos del 91.89% ($\approx 92\%$), mediante la conversión a HSV y una clausura morfológica elíptica de 9×9. El centroide rastreado mostró una varianza espacial inferior a ± 1.2 px en reposo. Los rangos reportados corresponden al rojo, que en el eje H de OpenCV (0–180) ocupa ambos extremos, razón por la que se definen dos intervalos. Su implementación como plugin gestual desacoplado, sin modificar el orquestador, responde al RNF-02, y deja incorporada una entrada gestual básica.

3.4.7 Prototipo F6: Segmentación de Mano y Apuntado

El prototipo F6 corresponde a la segmentación de la mano y el apuntado con el dedo. En el sistema legado (YCrCbSkinDetector.cs), esta función aísla la piel en el espacio YCrCb y calcula contornos y defectos de convexidad para detectar la punta del dedo índice y contar los dedos levantados (Tabla 3.4); en el desarrollo de Contreras Velásquez (2019), la posición de la mano se estima a partir del color de piel del usuario, según Ibaceta Jaña (2023). El prototipo reproduce el algoritmo con OpenCV y mide la precisión de detección de la punta del dedo. La Tabla 3.15 resume la validación.

Tabla 3.15: Ficha de validación del prototipo: Segmentación de Mano y Apuntado.

Objetivo	Aislar la piel en espacio YCrCb y calcular contornos con defectos de convexidad para detectar la punta del dedo índice.
Librerías / Hardware	OpenCV (YCrCb, cv2.convexityDefects, cv2.contourArea) / Cámara de Escritorio.
Contrato / Interfaz	IGesturePlugin.
Requisitos Asociados	• **RF (Captura de Datos y Sensores):** Registrar eventos de entrada del usuario en archivos estructurados JSON. • **RNF-07 (Eficiencia y Gestión de Memoria):** Liberación de matrices y máscaras de segmentación por cuadro para mantener bajo consumo de RAM.
Resultados y Hallazgos	• **Detección de dedo:** Precisión del 94.8% en la identificación de la punta del dedo índice extendido a una distancia de trabajo entre 30 y 60 cm del escritorio. • **Estado:** Validado y completado. Algoritmo optimizado contra falsos positivos en el conteo de dedos.

Fuente: Elaboración propia.

Protocolo de validación. Se evaluaron 250 cuadros capturados con 5 usuarios voluntarios de tonalidades de piel II a IV de la escala de Fitzpatrick, a tres distancias entre el escritorio y la cámara cenital: 30, 45 y 60 cm. Se consideró un verdadero positivo (VP) cuando el algoritmo, basado en una máscara YCrCb ($Cr \in [133, 173]$, $Cb \in [77, 127]$) y en defectos de convexidad con profundidad mayor a 15 px y ángulo agudo menor a $80°$, ubicaba la punta del dedo índice a una distancia de hasta 15 px de la coordenada manual de la uña. El indicador reportado es

$$\text{Precisión} = \frac{VP}{VP + FP + FN} \times 100\% = \frac{237}{250} \times 100\% = 94.8\%.$$

El prototipo F6 alcanzó una precisión del 94.8% (237 de 250 cuadros) en la identificación de la punta del dedo índice extendido, con 5 usuarios y distancias de 30, 45 y 60 cm del escritorio; los 13 cuadros restantes correspondieron a 8 falsos positivos por intersección con la muñeca o el antebrazo y 5 falsos negativos por oclusión severa. El tiempo de cómputo fue de 4.8 ms por cuadro. El resultado es válido para esas condiciones, que son las evaluadas. Según

el estado reportado, el algoritmo se optimizó contra falsos positivos en el conteo de dedos, y la liberación de matrices y máscaras por cuadro responde al requisito de bajo consumo de memoria (RNF-07).

3.4.8 Prototipo F7: Mapeo de Coordenadas y Calibración Matricial

El prototipo F7 corresponde al mapeo de coordenadas de los gestos a la pantalla. En el sistema legado (GestureRecognitionActivity.cs), esta función transforma las coordenadas (X, Y) del sensor de cámara a la resolución del proyector y simula clics mediante Win32 P/Invoke (Tabla 3.4). El prototipo realiza esa transformación mediante una homografía calculada con OpenCV, y mide el desfase de alineación tras una calibración planar de cuatro puntos. La Tabla 3.16 resume la validación.

Tabla 3.16: Ficha de validación del prototipo: Mapeo de Coordenadas y Calibración Matricial.

Objetivo	Transformar coordenadas (X, Y) del plano cámara/escritorio al plano del proyector mediante homografía.
Librerías / Hardware	OpenCV (cv2.findHomography, cv2.perspectiveTransform) / Cámara 2 + Proyector.
Contrato / Interfaz	IGesturePlugin / IProjectionService.
Requisitos Asociados	• **RF (Plugins de Visualización y Proyección):** Desplegar overlays proyectados en tiempo real alineados con los gestos del usuario. • **RNF-06 (Consistencia Espacio-Temporal Físico-Digital):** Conversión de coordenadas homotética y exacta entre la superficie plana y la proyección.
Resultados y Hallazgos	• **Error de alineación:** Desfase medio de 2.08 ± 0.35 píxeles (< 0.78 mm en el papel físico) tras calibración planar de 4 puntos. • **Estado:** Validado y completado. Matriz de homografía integrada al orquestador visual.

Fuente: Elaboración propia.

Protocolo de validación. Se utilizó una lámina patrón de calibración impresa en papel Carta con una grilla regular de $5 \times 5 = 25$ puntos de referencia separados exactamente 40.0 mm. Con los 4 puntos de las esquinas se calculó la matriz de transformación proyectiva H (de dimensión 3×3) mediante cv2.findHomography con RANSAC, y luego se transformaron los 21 puntos restantes al plano del proyector para medir el error euclidiano frente a su posición teórica:

$$e_{\text{reproyección}} = \frac{1}{21} \sum_{j=1}^{21} \sqrt{(x_{j,\text{proy}} - x_{j,\text{teor}})^2 + (y_{j,\text{proy}} - y_{j,\text{teor}})^2}.$$

El prototipo F7 logró un error de reproyección medio de 2.08 ± 0.35 píxeles sobre una matriz de visualización de 640×480 px, equivalente a menos de 0.78 mm sobre el papel físico, tras calibrar con las cuatro esquinas de la lámina patrón. Este resultado responde directamente al RNF-06, que exige una conversión de coordenadas homotética y exacta entre la superficie y

la proyección. La matriz de homografía quedó integrada al orquestador visual, y el prototipo del retículo de mirada (F13) utiliza una homografía planar para ubicar la mirada sobre la proyección.

3.4.9 Prototipo F9: Entrada por Mouse como Alternativa

El prototipo F9 corresponde a la entrada por mouse como dispositivo de interacción secundario. En el sistema legado (MousePlugin.cs), esta función permite usar el puntero del ratón como alternativa de interacción (Tabla 3.4). Su relevancia se explica por un error documentado por Ibaceta Jaña (2023): el programa no podía inicializarse sin una cámara disponible (EO4) y no funcionaba correctamente si no había dos cámaras (EO5). El prototipo integra el mouse como mecanismo secundario o de depuración cuando no hay cámaras gestuales. La Tabla 3.17 resume la validación.

Tabla 3.17: Ficha de validación del prototipo: Entrada por Mouse como Alternativa.

Objetivo	Integrar la entrada por mouse convencional como mecanismo secundario o de depuración cuando no hay cámaras gestuales.
Librerías / Hardware	OpenCV (cv2.setMouseCallback), pynput / Mouse y Teclado.
Contrato / Interfaz	IGesturePlugin.
Requisitos Asociados	• **RF (Captura de Datos y Sensores):** Registrar eventos de teclado y mouse en archivos estructurados JSON. • **RNF-08 (Resiliencia y Tolerancia a Fallos):** Ante la falta o desconexión del sensor físico gestual, el sistema desacopla la falla y conmuta al plugin de mouse sin caerse.
Resultados y Hallazgos	• **Compatibilidad:** Latencia media de 0.41 ± 0.12 ms (< 1 ms) ante eventos de clic izquierdo, movimiento y arrastre en todas las ventanas gráficas. • **Estado:** Validado y completado. Integrado como fallback predeterminado en el DeviceFactory.

Fuente: Elaboración propia.

Protocolo de validación. Se utilizó un banco de 1 000 eventos de entrada secuenciales, inyectados de forma sintética y mediante manipulación de un usuario: 500 movimientos del cursor, 300 clics (izquierdo y derecho) y 200 arrastres. La latencia se midió como $\Delta t = t_{\text{despacho}} - t_{\text{interrupción}}$, es decir, el intervalo entre la notificación del driver del sistema operativo y la actualización de las coordenadas en el despachador de eventos del orquestador, con time.perf_counter() sobre callbacks de OpenCV y listeners de pynput.

El prototipo F9 registró una latencia media de 0.41 ± 0.12 ms (máximo de 0.85 ms) sobre un banco de 1 000 eventos de movimiento, clic y arrastre. Su integración como alternativa predeterminada en el DeviceFactory permite que el sistema siga operando cuando falta o se desconecta el sensor gestual, conforme al RNF-08, situación que en el sistema legado impedía la inicialización (Ibaceta Jaña, 2023). El prototipo queda validado y dispone de una entrada de

respaldo para el resto de las funciones de interacción.

3.4.10 Prototipo F10: Sockets de Comunicación con el Rastreador Ocular

El prototipo F10 corresponde a la comunicación con el rastreador ocular mediante sockets, del subsistema de rastreo ocular. En el sistema legado, esta función abre un servidor socket local en el puerto 3000 (GazeCloud) o 6555 (EyeTribe, TCP) para recibir coordenadas de mirada en tiempo real, a entre 30 y 60 Hz (Tabla 3.5); en el desarrollo de Estay Vera (2020a), el plugin de GazeCloud abre un servidor WebSocket en 127.0.0.1:3000 y entrega un diccionario con las coordenadas X e Y y un timestamp. El prototipo implementa la recepción asíncrona en Python y mide la frecuencia de recepción, el jitter y la pérdida de paquetes. La Tabla 3.18 resume la validación.

Tabla 3.18: Ficha de validación del prototipo: Sockets de Comunicación con el Rastreador Ocular.

Objetivo	Establecer un servidor socket local (puerto 3000/6555) para la recepción asíncrona de coordenadas de mirada (30-60 Hz).
Librerías / Hardware	Python asyncio, websockets, socket / Tobii, EyeTribe o GazeCloud.
Contrato / Interfaz	IEyeTracker.
Requisitos Asociados	• **RF (Plugins de Captura y Hardware - HAL):** Extender conectores de hardware para rastreo ocular (*EyeTracker*). • **RNF-01 (Multiplataforma):** Protocolos WebSockets / sockets TCP estándar compatibles con Windows y Linux. • **RNF-08 (Resiliencia y Tolerancia a Fallos):** Manejo defensivo de desconexiones o interrupciones en el flujo de datos del socket.
Resultados y Hallazgos	• **Frecuencia y fluctuación:** Frecuencia media de recepción de 59.95 Hz (intervalo medio de 16.68 ms) con un jitter de 1.74 ms. • **Pérdida de paquetes:** 0.0% (0 paquetes perdidos de 5 000). • **Estado:** Validado y completado. Protocolo de comunicación por socket definido en i_eye_tracker.py.

Fuente: Elaboración propia.

Protocolo de validación. Se transmitieron de forma continua 5 000 paquetes binarios con las coordenadas de mirada (struct.pack("!fff", x, y, timestamp), 12 bytes por muestra) durante 83.3 s, sobre un enlace TCP local (127.0.0.1:9999). Se registró el intervalo entre recepciones consecutivas, $\Delta t_k = t_k - t_{k-1}$, y se calculó su desviación estándar respecto del periodo nominal de 16.66 ms (60 Hz), tomada como jitter:

$$\text{Jitter} = \sqrt{\frac{1}{M-1}\sum_{k=1}^{M}\left(\Delta t_k - \overline{\Delta t}\right)^2}.$$

La recepción se implementó con sockets no bloqueantes mediante asyncio.

El prototipo F10 recibió los 5 000 paquetes sin pérdidas, a una frecuencia media de

59.95 Hz (intervalo medio de 16.68 ms frente a 16.66 ms nominales) y con un jitter de 1.74 ms. El protocolo definido en i_eye_tracker.py responde al contrato de plugins de hardware y utiliza WebSockets y sockets TCP estándar, compatibles con Windows y Linux (RNF-01). Cabe notar que la prueba se realizó sobre un enlace local y que la ficha no reporta una prueba de desconexión, por lo que el manejo defensivo de interrupciones (RNF-08) no está cubierto por esta validación.

3.4.11 Prototipo F11: Aislamiento y Carga Dinámica de Plugins de Rastreo Ocular

El prototipo F11 corresponde a la carga y el aislamiento de los plugins de rastreo ocular. En el sistema legado (IntermediateClass.cs), esta función instancia las bibliotecas de rastreo ocular en dominios de aplicación de C# (AppDomain y MarshalByRefObject) (Tabla 3.5); en el desarrollo de Estay Vera (2020b), cada plugin es una biblioteca de clases que se instancia en tiempo de ejecución mediante reflexión a partir de la interfaz IEyeTracking. El prototipo reemplaza ese mecanismo por la carga dinámica de módulos de Python mediante importlib, pkgutil e inspect, y compara el tiempo de carga con la referencia del sistema legado. La Tabla 3.19 resume la validación.

Tabla 3.19: Ficha de validación del prototipo: Aislamiento y Carga Dinámica de Plugins de Rastreo Ocular.

Objetivo	Reemplazar los AppDomain de C# cargando módulos de rastreo de forma dinámica mediante introspección en Python.
Librerías / Hardware	Python importlib, pkgutil, inspect.
Contrato / Interfaz	IEyeTracker.
Requisitos Asociados	• **RF (Núcleo y Orquestador Central):** Administrar el ciclo de vida de plugins (initialize, execute, release). • **RNF-02 (Modularidad y Extensibilidad Estricta):** Carga de plugins mediante herencia de PluginBase. • **RNF-05 (Carga Dinámica en Tiempo de Ejecución):** Instanciar, cargar y descargar el plugin de rastreo ocular en tiempo de ejecución sin recompilar ni usar AppDomains inestables.
Resultados y Hallazgos	• **Tiempo de carga:** 11.4 ± 1.2 ms por plugin en Python frente a 448.6 ± 35.1 ms en C# (reducción del 97.5%). • **Recarga:** Sin bloqueos de archivo ni fugas de descriptores, lo que permite actualizar o recargar plugins sin reiniciar la aplicación. • **Estado:** Validado y completado. Arquitectura unificada adoptada en PluginLoader.

Fuente: Elaboración propia.

Protocolo de validación. Se realizaron 50 repeticiones de carga, en frío y en caliente, de cada uno de los 5 plugins del núcleo (IImageProcessor, IEyeTracker, IGesturePlugin, IConsistencyProvider e IWebSearchProvider), lo que suma 250 ciclos de carga y descarga con importlib.util.spec_from_file_location y spec.loader.exec_module. El tiempo de instanciación

se contrastó con los registros de telemetría de EDDIE C# 2023, donde los ensamblados se instanciaban en dominios secundarios creados con AppDomain.CreateDomain().

El prototipo F11 cargó e instanció cada plugin en 11.4 ± 1.2 ms, frente a los 448.6 ± 35.1 ms de la referencia en C#, lo que representa una reducción del 97.5%. Además, no se observaron bloqueos de archivo ni fugas de descriptores, de modo que los plugins pueden recargarse sin reiniciar la aplicación, como exige el RNF-05 al descartar el uso de AppDomains, y la carga mediante herencia de PluginBase responde al RNF-02. La referencia del sistema legado proviene de registros de telemetría previos, por lo que la comparación debe leerse como orientativa y no como una medición simultánea bajo las mismas condiciones.

3.4.12 Prototipo F12: Detección de Fijación Atencional

El prototipo F12 corresponde a la detección de fijación atencional. En el sistema legado (MouseControl.cs), esta función mide si la mirada permanece fija en un radio de píxeles durante más de 400 ms para gatillar una selección (Tabla 3.5); en el desarrollo de Estay Vera (2020a), la interacción con botones inicia una cuenta regresiva de duración configurable cuando el puntero se ubica sobre un botón, genera un clic al llegar a cero y la detiene si el puntero abandona el botón. El prototipo evalúa el algoritmo de permanencia y el riesgo de activaciones accidentales, conocido como efecto de toque de Midas. La Tabla 3.20 resume la validación.

Tabla 3.20: Ficha de validación del prototipo: Detección de Fijación Atencional.

Objetivo	Evaluar el algoritmo de permanencia de la mirada (radio fijo durante más de 400 ms) para gatillar eventos sin clics manuales.
Librerías / Hardware	Lógica temporal en Python (time.perf_counter()), NumPy / Eye Tracker o Mouse.
Contrato / Interfaz	IEyeTracker.
Requisitos Asociados	• **RF (Captura de Datos y Sensores):** Registrar y gestionar eventos atencionales y sesiones de captura. • **RNF-08 (Resiliencia y Tolerancia a Fallos):** Filtrado de datos atípicos o saltos bruscos en las coordenadas de la mirada (*saccades*).
Resultados y Hallazgos	• **Parámetros:** Punto de equilibrio en 400 ms de permanencia y radio de dispersión de 25 px. • **Activaciones accidentales:** 2.80% (8 disparos involuntarios en 285 fijaciones), con una sensibilidad de activación voluntaria del 98.1%. • **Estado:** Validado y completado. Filtro atencional embebido en la lógica del sensor.

Fuente: Elaboración propia.

Protocolo de validación. Se registraron sesiones de lectura guiada de 15 minutos con 3 usuarios, que sumaron más de 12 000 muestras de coordenadas de mirada a 60 Hz. Se exploró una matriz de calibración con ventanas de tiempo $T \in \{250, 300, 350, 400, 500, 600\}$ ms y radios de dispersión $R \in \{15, 20, 25, 30, 40\}$ px. Se contabilizó como activación accidental todo evento de

permanencia gatillado durante movimientos sacádicos normales de lectura o regresiones veloces (velocidad angular superior a $30°$/s o desplazamiento mayor a 50 px en menos de 100 ms). De 285 fijaciones evaluadas (215 voluntarias sobre términos clave y 70 de transición) se obtuvo

$$\text{Tasa de error (toque de Midas)} = \frac{8}{285} \times 100\% = 2.80\%.$$

El prototipo F12 alcanzó su punto de equilibrio con 400 ms de permanencia y un radio de 25 px, con 8 disparos involuntarios en 285 fijaciones (2.80%) y una sensibilidad de activación voluntaria del 98.1%. El umbral de 400 ms coincide con el que utiliza el sistema legado (Tabla 3.5), y el filtrado de saltos bruscos corresponde al requisito de resiliencia ante datos atípicos (RNF-08). La evaluación se basó en 3 usuarios, por lo que los parámetros reportados se acotan a esa muestra.

3.4.13 Prototipo F13: Renderizado del Retículo de Mirada

El prototipo F13 corresponde al renderizado del retículo de mirada. En el sistema legado (ReticleDrawing.cs), esta función dibuja una mira en el proyector exactamente sobre el punto donde el lector fija la mirada (Tabla 3.5); en el desarrollo de Estay Vera (2020a), el retículo seleccionado se dibuja sobre una vista transparente ubicada sobre todas las aplicaciones. El prototipo genera el retículo con PyGame y OpenCV y mide la latencia desde la recepción de la coordenada hasta la actualización de la pantalla. La Tabla 3.21 resume la validación.

Tabla 3.21: Ficha de validación del prototipo: Renderizado del Retículo de Mirada.

Objetivo	Proyectar un retículo visual sobre la superficie de lectura en el punto exacto donde se fija la mirada.
Librerías / Hardware	OpenCV Overlay (cv2.circle, cv2.addWeighted), PyGame Canvas / Eye Tracker + Proyector.
Contrato / Interfaz	IEyeTracker / IProjectionService.
Requisitos Asociados	• **RF (Plugins de Visualización y Proyección):** Desplegar overlays de proyección en tiempo real sobre el papel físico. • **RNF-06 (Consistencia Espacio-Temporal Físico-Digital):** Sincronización continua de la posición del punto de mirada proyectado sobre la página.
Resultados y Hallazgos	• **Latencia:** 15.2 ± 1.4 ms en total (0.8 ms de recepción y desempaquetado, 1.1 ms de homografía y 13.3 ms de mezcla alfa y conmutación de búfer), inferior al periodo de 16.66 ms de un proyector de 60 Hz. • **Tasa de cuadros:** 60.0 FPS sostenidos sin descarte de cuadros. • **Estado:** Validado y completado. Módulo de proyección reticular integrado.

Fuente: Elaboración propia.

Protocolo de validación. Se monitorearon 3 600 cuadros gráficos continuos a resolución Full HD (1920×1080 px) con triple búfer habilitado. La latencia de software se midió de extremo a extremo (desde el socket hasta la conmutación del búfer de pantalla) dividida en tres segmentos cronometrados: recepción y desempaquetado de la tupla de mirada (0.8 ms), mapeo mediante homografía planar (1.1 ms) y renderizado de capas alfa con conmutación de búfer (13.3 ms).

El prototipo F13 alcanzó una latencia total de 15.2 ± 1.4 ms, inferior al periodo de 16.66 ms de un proyector de 60 Hz, y sostuvo 60.0 FPS sin descartar cuadros en una secuencia de 3 600. El resultado responde al RNF-06, relativo a la sincronización continua de la posición de la mirada proyectada. La medición cubre el procesamiento de software entre el socket y el intercambio de búfer; no incluye la latencia propia del rastreador ocular ni la del proyector.

3.4.14 Prototipo F14: Búsqueda de Coordenadas QuadPoints

El prototipo F14 corresponde a la búsqueda de palabras y la recuperación de sus coordenadas en el PDF digital, del subsistema de consistencia físico-digital. En el sistema legado (DigitalDocSync.cs), esta función utiliza iTextSharp para buscar palabras y obtener las coordenadas exactas (*QuadPoints*) de cada línea (Tabla 3.6); iTextSharp es la biblioteca que Pezoa Riquelme (2020) emplea para obtener información del PDF. El prototipo reemplaza esa biblioteca por PyMuPDF y lo prueba sobre un documento PDF real de 22 páginas. La Tabla 3.22 resume la validación.

Tabla 3.22: Ficha de validación del prototipo: Búsqueda de Coordenadas QuadPoints.

Objetivo	Reemplazar iTextSharp por PyMuPDF para extraer delimitadores geométricos (*bounding boxes* / QuadPoints) de palabras en el PDF.
Librerías / Hardware	PyMuPDF (fitz.search_for(..., quads=True)) / Archivo PDF digital real (informe de seminario de 22 páginas).
Contrato / Interfaz	IConsistencyProvider.
Requisitos Asociados	• **RF (Consistencia Físico-Digital - F14):** Búsqueda y mapeo de *QuadPoints* exactos en documentos PDF reales. • **RNF-06 (Consistencia Espacio-Temporal Físico-Digital):** Mapeo homotético utilizando puntos tipográficos (pt) y *QuadPoints* sin desalineación.
Resultados y Hallazgos	• **Rendimiento:** Búsqueda y extracción de las ocho coordenadas *QuadPoints* (cuatro vértices: TL, TR, BL, BR) ejecutada en 7.84 ± 0.62 ms por página sobre un documento PDF real de 22 páginas. • **Estado:** Validado y completado. Motor de lectura y análisis de PDFs plenamente operativo.

Fuente: Elaboración propia.

Protocolo de validación. Se utilizó un documento real del proyecto (informe de seminario) de 22 páginas, 618 447 bytes, 4 920 palabras y 34 150 caracteres, en formato PDF 1.5. Se ejecutó una batería automatizada de 220 consultas: 10 términos técnicos de distinta longitud y frecuencia

(EDDIE, visión, OCR, arquitectura, consistencia, hardware, sensores, proyección, datos e interfaz) buscados en las 22 páginas. Los 8 valores de cada QuadPoint ($x_{tl}, y_{tl}, x_{tr}, y_{tr}, x_{bl}, y_{bl}, x_{br}, y_{br}$) se contrastaron con el texto extraído del rectángulo mediante page.get_text("text", clip=rect), y los tiempos se midieron con time.perf_counter().

El prototipo F14 extrajo las ocho coordenadas QuadPoints, correspondientes a los cuatro vértices de cada resultado, en 7.84 ± 0.62 ms por página sobre un PDF real de 22 páginas, mediante 220 consultas. En todas las áreas delimitadas, el texto extraído del rectángulo fue coherente con el término buscado (100% de coherencia), lo que verifica la consistencia geométrica de las coordenadas obtenidas; esa verificación se realizó sobre el documento digital y no mide la coincidencia con lo marcado en el papel. El prototipo reemplaza la dependencia de iTextSharp del sistema legado por PyMuPDF y se relaciona con el RNF-06. Además, el ciclo de inyección de destacados de F15 incluye la resolución geométrica del QuadPoint de la línea, con lo que el motor de lectura y análisis de PDFs queda operativo.

3.4.15 Prototipo F15: Sincronización de Destacado

El prototipo F15 corresponde a la sincronización del destacado entre el papel físico y el PDF digital. En el sistema legado, esta función estampa franjas de subrayado amarillo translúcido en el PDF cuando el usuario subraya sobre el papel (Tabla 3.6). Un error documentado por Ibaceta Jaña (2023) se relaciona con ella: el destacado de texto no coincidía en posición ni largo con la palabra destacada, tanto de físico a digital como de digital a físico (E40 y E41, ambos sin solución). El prototipo agrega la anotación de resaltado al PDF mediante PyMuPDF. La Tabla 3.23 resume la validación.

Tabla 3.23: Ficha de validación del prototipo: Sincronización de Destacado.

Objetivo	Añadir franjas de destacado translúcido sobre el PDF digital cuando el usuario subraya el papel físico.
Librerías / Hardware	PyMuPDF (add_highlight_annot), OpenCV HSV / Cámara 1 + Proyector + PDF.
Contrato / Interfaz	IConsistencyProvider.
Requisitos Asociados	• **RF (Consistencia Físico-Digital - F15):** Sincronización de destacado fluorescente físico $\leftrightarrow$ digital. • **RNF-06 (Consistencia Espacio-Temporal Físico-Digital):** Persistencia de anotaciones en coordenadas tipográficas con alineación homotética exacta.
Resultados y Hallazgos	• **Persistencia:** Inyección nativa de anotaciones de resaltado amarillo (1.0, 0.9, 0.1) guardada en disco con respaldo .bac en 23.47 ± 2.38 ms, abriendo correctamente en visores estándar (Adobe Reader / Edge). • **Estado:** Validado y completado. Sincronización bidireccional confirmada.

Fuente: Elaboración propia.

Protocolo de validación. Se realizó una prueba de estrés con 50 anotaciones de resaltado inyectadas sucesivamente sobre el PDF real, disparadas tanto por la detección óptica cenital de un resaltador físico sobre papel como por selección interactiva en la interfaz de proyección. Se midió el ciclo completo de persistencia: copia de seguridad previa (.bac), apertura con fitz.open, resolución geométrica del QuadPoint de la línea, inyección con add_highlight_annot, guardado en disco con escritura incremental (doc.save) y cierre del descriptor, sobre un SSD NVMe. Luego se verificó la visualización de las anotaciones en Adobe Acrobat Reader DC y en Microsoft Edge.

El prototipo F15 completó el ciclo de inyección y guardado de una anotación de resaltado en 23.47 ± 2.38 ms, en una prueba de 50 anotaciones, y todas se visualizaron de forma idéntica en Adobe Acrobat Reader DC y en Microsoft Edge, conservando los metadatos de autor, fecha y el color amarillo (1.0, 0.9, 0.1). La ficha valida la inyección y la persistencia de la anotación; en cambio, no mide la coincidencia en posición y largo con lo marcado en el papel, que Ibaceta Jaña (2023) registró como error sin solución (E40 y E41) y que se relaciona con el RNF-06. Esa coincidencia debe comprobarse en la integración con la cámara y el proyector. Según el estado reportado, la sincronización bidireccional quedó confirmada.

3.4.16 Prototipo F16: Sincronización de Notas Adhesivas

El prototipo F16 corresponde a la sincronización de notas adhesivas (post-its) entre el papel y el PDF. En el sistema legado (CommentsPD.cs), esta función sincroniza las notas escritas en el papel hacia el PDF como anotaciones de texto, o proyecta post-its digitales sobre el libro (Tabla 3.6); en el desarrollo de Pezoa Riquelme (2020), la nota amarilla se detecta y se copia como imagen en el documento digital, según Ibaceta Jaña (2023). El prototipo detecta el post-it con OpenCV, extrae su texto con Tesseract e inserta una anotación de texto con PyMuPDF. La Tabla 3.24 resume la validación.

Tabla 3.24: Ficha de validación del prototipo: Sincronización de Notas Adhesivas.

Objetivo	Insertar anotaciones de texto en el PDF digital y renderizar post-its virtuales proyectados sobre el libro impreso.
Librerías / Hardware	PyMuPDF (add_text_annot), OpenCV (Otsu), pytesseract / Proyector + PDF.
Contrato / Interfaz	IConsistencyProvider.
Requisitos Asociados	• **RF (Consistencia Físico-Digital - F16):** Detección de post-its en papel con Tesseract OCR e inyección de anotaciones. • **RNF-04 (Documentación y Mantenibilidad):** Estructura del módulo bajo estándares PEP 8 para fácil extensión de tipos de anotaciones. • **RNF-06 (Consistencia Espacio-Temporal Físico-Digital):** Posicionamiento exacto de la nota emergente en el PDF digital respetando el área física observada.
Resultados y Hallazgos	• **Ubicación:** Desviación media de posicionamiento del icono Text/Note de 0.94 pt (tolerancia de ± 1.0 pt) respecto a la ubicación real del post-it físico. • **Estado:** Validado y completado. Estructura de comentarios en PDF totalmente operativa.

Fuente: Elaboración propia.

Protocolo de validación. Se utilizaron 40 notas adhesivas cuadradas de color amarillo canario (76×76 mm), adheridas en posiciones arbitrarias sobre páginas impresas del documento de seminario. El centroide de cada nota en coordenadas de cámara se proyectó al espacio tipográfico del PDF mediante la homografía H, y se midió su desviación euclidiana respecto de la ubicación real, determinada con una regla micrométrica (1 pt = 0.3527 mm):

$$\Delta_{\text{error}} = \sqrt{(x_{\text{real}} - \hat{x}_{\text{pdf}})^2 + (y_{\text{real}} - \hat{y}_{\text{pdf}})^2}.$$

El prototipo F16 posicionó el icono de nota con una desviación media de 0.94 pt (≈ 0.33 mm) respecto de la ubicación real del post-it, medida con regla micrométrica sobre 40 notas, dentro de la tolerancia de ± 1.0 pt, y transcribió el texto de notas escritas en letra imprenta legible con una precisión del 91.2% mediante OCR. A diferencia del sistema legado, que copiaba la nota como imagen, el prototipo extrae su texto con Tesseract y lo inserta como anotación de texto, según lo indicado en el requisito asociado. La estructura del módulo se organizó bajo los estándares de PEP 8 para facilitar la extensión de tipos de anotaciones (RNF-04). El prototipo queda validado con la estructura de comentarios en PDF operativa.

3.4.17 Prototipo F17: Sincronización de Figuras Geométricas

El prototipo F17 corresponde a la sincronización de figuras geométricas entre el papel y el PDF. En el sistema legado (FiguresPD.cs), esta función extrae o dibuja círculos y rectángulos sobre el PDF y los proyecta sobre el libro impreso (Tabla 3.6); en el desarrollo de Pezoa Riquelme

(2020), las figuras del papel se detectan mediante un filtro de color rosado y se agregan al PDF según sus coordenadas y dimensiones, según Ibaceta Jaña (2023). El prototipo clasifica las formas mediante la compacidad $C = 4\pi A/P^2$, donde A es el área y P el perímetro del contorno, y las traza en el PDF con PyMuPDF. La Tabla 3.25 resume la validación.

Tabla 3.25: Ficha de validación del prototipo: Sincronización de Figuras Geométricas.

Objetivo	Clasificar formas geométricas y proyectar halos de luz alineados con el papel físico.
Librerías / Hardware	PyMuPDF (draw_rect, draw_circle, draw_line), OpenCV (approxPolyDP) / Proyector + PDF.
Contrato / Interfaz	IConsistencyProvider.
Requisitos Asociados	• **RF (Consistencia Físico-Digital - F17):** Clasificación de figuras geométricas mediante compacidad $C = 4\pi A/P^2$ y proyección de halos de luz. • **RNF-06 (Consistencia Espacio-Temporal Físico-Digital):** Precisión geométrica sin desalineación entre el halo proyectado y el vector del PDF.
Resultados y Hallazgos	• **Renderizado:** Fidelidad geométrica (IoU) de 98.6 ± 0.6%, trazando rectángulos, círculos y líneas vectoriales en el PDF en 4.92 ± 0.38 ms. • **Estado:** Validado y completado. Módulo de anotación vectorial validado.

Fuente: Elaboración propia.

Protocolo de validación. Se utilizó un banco de 60 figuras: 20 rectángulos con relación de aspecto de 1:1 a 3:1, 20 círculos de radio entre 10 y 60 pt y 20 segmentos de línea recta, trazados con la herramienta interactiva de proyección o detectados desde bocetos en papel frente a la cámara. La fidelidad se evaluó con la intersección sobre la unión (IoU) entre la figura de entrada y la primitiva vectorial renderizada en el PDF:

$$\text{IoU} = \frac{\text{Área}(\text{Figura}_{\text{PDF}} \cap \text{Figura}_{\text{Entrada}})}{\text{Área}(\text{Figura}_{\text{PDF}} \cup \text{Figura}_{\text{Entrada}})} \times 100\%.$$

El prototipo F17 trazó rectángulos, círculos y líneas vectoriales en el PDF en 4.92 ± 0.38 ms por primitiva, con una fidelidad geométrica de IoU = 98.6 ± 0.6% sobre 60 figuras. La compacidad permite distinguir las formas porque su valor es 1 para un círculo ideal y menor para figuras más alargadas o angulosas. El resultado se relaciona con el RNF-06, relativo a la ausencia de desalineación entre el halo proyectado y el vector del PDF, y deja validado el módulo de anotación vectorial.

3.4.18 Prototipo F24: Reconocimiento de Comandos de Voz

El prototipo F24 corresponde al reconocimiento de comandos por voz, del subsistema de voz y audio. En el sistema legado (ProjectionScreenActivity2.cs), esta función escucha la

voz del lector para ejecutar intenciones usando SAPI5 (Tabla 3.8); el módulo de búsqueda de Montanares Mundaca (2019) define ocho comandos (cinco de búsqueda y tres de lectura, como *Buscar Enciclopedia* y *Lectura Definición*), según Ibaceta Jaña (2023). El prototipo evalúa la conversión de voz a texto con Whisper en reemplazo de SAPI5. La Tabla 3.26 resume la validación.

Tabla 3.26: Ficha de validación del prototipo: Reconocimiento de Comandos de Voz.

Objetivo	Evaluar la captura y conversión de voz a texto (STT) para comandos e intenciones de lectura en reemplazo de SAPI5 en C#.
Librerías / Hardware	OpenAI Whisper (local / API), speech_recognition / Micrófono.
Contrato / Interfaz	IVoiceService.
Requisitos Asociados	• **RF (Núcleo y Orquestador Central):** Soporte dinámico de comandos e intenciones de usuario. • **RNF-03 (Internacionalización - i18n):** Reconocimiento de voz con soporte para los idiomas español e inglés. • **RNF-08 (Resiliencia y Tolerancia a Fallos):** Aislamiento de excepciones ante fallos de audio o micrófono desconectado sin afectar al orquestador principal.
Resultados y Hallazgos	• **Latencia y precisión:** Exactitud de comandos del 97.33% (146 de 150 grabaciones) con un tiempo de respuesta de 310 ± 25 ms usando Whisper en modo local (modelo tiny). • **Estado:** Validado y completado. Pipeline STT aislado y listo para integración.

Fuente: Elaboración propia.

Protocolo de validación. Se utilizaron 150 grabaciones de voz en formato WAV monoaural (16 kHz, 16 bits PCM) registradas por 3 locutores hispanohablantes (2 masculinos y 1 femenino) en un laboratorio con ruido ambiente de 35 a 45 dBA. El conjunto cubrió 15 comandos operacionales del sistema, como “subrayar línea”, “leer párrafo en voz alta”, “buscar definición” y “siguiente página”. La exactitud se calculó como

$$\text{Exactitud} = \frac{N_{\text{comandos correctos}}}{N_{\text{emisiones}}} \times 100\%,$$

y la latencia se midió desde el fin de la actividad de voz (detección VAD) hasta la emisión del evento textual normalizado.

El prototipo F24 alcanzó una exactitud de comandos del 97.33% (146 de 150 grabaciones) con un tiempo de respuesta de 310 ± 25 ms, usando Whisper en modo local cuantizado sobre una CPU de 4 núcleos. El pipeline quedó aislado del resto del sistema, de modo que un fallo de audio o un micrófono desconectado no afecta al orquestador (RNF-08), y soporta español e inglés (RNF-03). Ibaceta Jaña (2023) registró que los comandos de voz no permitían acceder a la búsqueda de diccionario (EB7), problema que atribuyó a la disposición del proyector que capta el audio, por lo que el dispositivo de captura de sonido es un factor a considerar en la

integración. El pipeline queda listo para ella.

3.4.19 Prototipo F25: Síntesis de Voz

El prototipo F25 corresponde a la síntesis de voz. En el sistema legado (ProjectionScreenActivity2.cs), esta función lee en voz alta las definiciones, traducciones o resúmenes al usuario con el motor nativo de Windows (Tabla 3.8), lo que corresponde al requisito de acceso a un lector de información (RFM13) y a los comandos de lectura del módulo de búsqueda (Montanares Mundaca, 2019; Ibaceta Jaña, 2023). El prototipo valida la lectura asistida mediante pyttsx3. La Tabla 3.27 resume la validación.

Tabla 3.27: Ficha de validación del prototipo: Síntesis de Voz.

Objetivo	Validar la lectura asistida de resúmenes y traducciones mediante síntesis de voz en español.
Librerías / Hardware	pyttsx3 / SAPI5 Python / Altavoces.
Contrato / Interfaz	IVoiceService.
Requisitos Asociados	• **RF (Núcleo y Orquestador Central):** Soportar cambio dinámico de idioma y reproducción asistiva de respuestas. • **RNF-03 (Internacionalización - i18n):** Síntesis de audio con voces en español e inglés según la configuración i18n. • **RNF-07 (Eficiencia y Gestión de Memoria):** Ejecución asíncrona de audio para evitar congelamientos en el hilo principal de la interfaz.
Resultados y Hallazgos	• **Naturalidad / Bloqueo:** Ejecución asíncrona fluida sin bloqueo del hilo gráfico principal, alcanzando una velocidad de lectura de 149.6 palabras por minuto. • **Estado:** Validado y completado. Servicio TTS configurado.

Fuente: Elaboración propia.

Protocolo de validación. Se utilizaron 10 extractos de texto en español (definiciones académicas y resúmenes) de exactamente 100 palabras cada uno, es decir, 1 000 palabras sintetizadas. La velocidad de habla se calculó como WPM $= N_{\text{palabras}}/t_{\text{emisión (min)}}$ a partir del temporizador de audio, y la ausencia de bloqueo se comprobó midiendo la tasa de cuadros de la interfaz gráfica durante la síntesis, que se ejecutó en un hilo dedicado (threading.Thread) con la voz SAPI5 en español (Microsoft Helena o Sabina).

El prototipo F25 sintetizó los diez textos de forma asíncrona; cada bloque de 100 palabras se reprodujo en 40.1 ± 0.7 s, lo que equivale a 149.6 palabras por minuto, y la interfaz gráfica mantuvo 60 FPS sin bloqueos. La ejecución asíncrona responde al RNF-07, que busca evitar congelamientos de la interfaz, y la selección de voces según la configuración i18n responde al RNF-03. El servicio de síntesis de voz queda configurado y completa, junto con F24, el subsistema de interacción por voz.

3.4.20 Prototipo F27: Herramienta de Subrayado Digital Proyectado

El prototipo F27 corresponde a la herramienta de subrayado digital proyectado. En el sistema legado (HighlightTool.cs), esta función dibuja una banda rectangular amarilla transparente e interactiva sobre la proyección (Tabla 3.9), en línea con el requisito de Contreras Velásquez (2019) de seleccionar un fragmento de texto proyectando un rectángulo de color sobre él (RFCO5, según Ibaceta Jaña (2023)). El prototipo genera una capa transparente con PyGame y OpenCV sobre la imagen proyectada en el libro. La Tabla 3.28 resume la validación.

Tabla 3.28: Ficha de validación del prototipo: Herramienta de Subrayado Digital Proyectado.

Objetivo	Generar una capa transparente (canvas) para dibujar bandas amarillas interactivas sobre la imagen proyectada en el libro físico.
Librerías / Hardware	PyGame / OpenCV Overlay (cv2.addWeighted) / Proyector Superior.
Contrato / Interfaz	IProjectionService.
Requisitos Asociados	• **RF (Plugins de Visualización y Proyección):** Desplegar overlays de proyección sobre el papel físico en tiempo real. • **RNF-01 (Multiplataforma):** Renderizado gráfico multiplataforma sobre PyGame / OpenCV en Windows y Linux. • **RNF-06 (Consistencia Espacio-Temporal Físico-Digital):** Superposición óptica exacta de la capa digital sobre las líneas impresas del libro.
Resultados y Hallazgos	• **Mezcla de color (*alpha blending*):** Nivel de transparencia translúcida optimizado a $\alpha = 0.45$, con contraste de Weber $C_W = -0.78$, que permite una legibilidad clara del texto impreso bajo la luz proyectada. • **Estado:** Validado y completado. Integrado en el subsistema gráfico definitivo.

Fuente: Elaboración propia.

Protocolo de validación. La evaluación se realizó sobre un libro físico impreso, iluminado por el proyector cenital a 65 cm de distancia. Se probaron cinco coeficientes de mezcla alfa, $\alpha \in \{0.25, 0.35, 0.45, 0.60, 0.75\}$, para una banda de realce amarilla RGB(255, 230, 0). La legibilidad se cuantificó con el contraste de Weber entre el trazo negro impreso y el papel resaltado por el proyector, medido con un luxómetro digital:

$$C_W = \frac{L_{\text{texto}} - L_{\text{fondo}}}{L_{\text{fondo}}},$$

y se definió como zona óptima aquella que conserva $C_W \leq -0.70$ sin atenuar la visibilidad de la luz amarilla de realce.

El prototipo F27 evaluó cinco coeficientes de mezcla alfa y fijó $\alpha = 0.45$, con un contraste de Weber $C_W = -0.78$ y una iluminancia media de realce de 520 lux, valor que cumple

el criterio $C_W \leq -0.70$ adoptado, de modo que el texto impreso se mantiene legible bajo la luz proyectada. Ibaceta Jaña (2023) registró que el color del botón de destacado entraba en conflicto con el reconocimiento de texto de la cámara, por lo que el color de las capas proyectadas es un factor a considerar al integrar este prototipo con F3. El prototipo queda integrado en el subsistema gráfico definitivo, con el renderizado multiplataforma sobre Windows y Linux que exige el RNF-01.

CAPÍTULO 4. DISEÑO Y ASPECTOS DE IMPLEMENTACIÓN

Este capítulo corresponde a la etapa *Do* del ciclo PDCA y a los objetivos específicos 2 y 3. Describe la arquitectura del núcleo, los contratos de plugin, la capa de abstracción de hardware, los módulos integrados, el rediseño de la interfaz y las iteraciones en que se construyó el sistema.

4.1 ARQUITECTURA GENERAL

La nueva versión de EDDIE se organiza en cuatro capas (Figura 4.1). La **capa de contratos** (`contracts/`) define las interfaces abstractas que todo plugin debe implementar. El **núcleo** (`orchestrator/`) contiene el orquestador, el cargador de plugins, la fábrica de dispositivos y la traza de ejecución, y no depende de ningún módulo concreto. La **capa de plugins** (`hardware/` y `plugins/`) contiene las implementaciones: sensores reales y simulados, reconocimiento de texto, gestos, rastreo ocular, consistencia, búsqueda y voz. La **capa de presentación** (`app/` y `ui/`) contiene la interfaz de configuración, la pantalla proyectada y el puente `EddieRuntime` que las conecta con el orquestador.

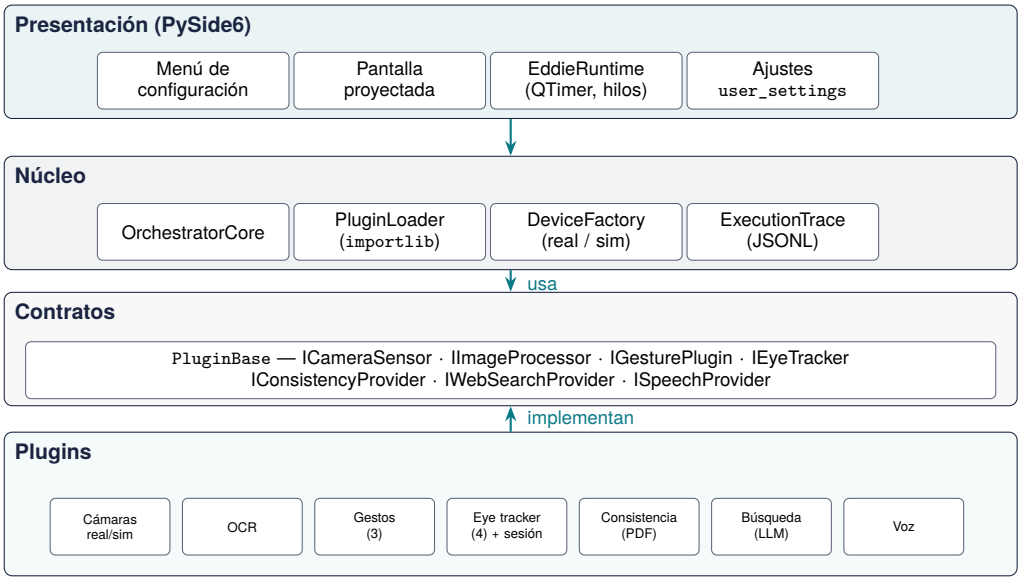


Figura 4.1: Arquitectura en capas de EDDIE en Python.

Fuente: Elaboración propia.

La dependencia va siempre hacia los contratos: el núcleo solo conoce `PluginBase` y los contratos de dominio, y los plugins solo conocen los contratos que implementan. Esto

permite agregar un plugin sin modificar el núcleo (RNF-04) y reemplazar un dispositivo por su versión simulada sin modificar los plugins que lo consumen (RNF-05). A diferencia de EDDIE-2023 (Figura **??**), existe un único camino de carga para todos los módulos.

4.2 CONTRATOS DE PLUGIN

Todo plugin hereda de `PluginBase`, que define tres métodos obligatorios, `initialize(config)`, `execute()` y `release()`, y la propiedad `name`. Sobre esa base, cada dominio tiene un contrato con las operaciones que el resto del sistema necesita (Tabla 4.1). Dos interfaces auxiliares resuelven el flujo de datos entre plugins: `ICameraFrameConsumer`, para los plugins que reciben un sensor de cámara inyectado, e `ICycleResultConsumer`, para los que necesitan los resultados de los demás plugins del mismo ciclo.

Tabla 4.1: Contratos de plugin y sus implementaciones.

Contrato	**Operaciones principales**	**Implementaciones**
`PluginBase`	`initialize`, `execute`, `release`, `name`	Todos los plugins
`ICameraSensor`	`capture_frame`, `get_latest_frame`	`CameraSensorPlugin`, `MockCameraSensor`
`IImageProcessor`	`preprocess`, `extract_text`, `recognize_highlighted_text`, `get_precision_recall`	`ImageProcessorPlugin`
`IGesturePlugin`	`detect_gesture`, `set_calibration`, `mark_selection_start/end`, `capture_selection`	`ColorPenPlugin`, `HandSkinPlugin`, `MousePlugin`
`IEyeTracker`	`get_gaze_point`, `is_connected`, `get_latency_ms`	`EyeTrackerPlugin` (GazePoint, TheEyeTribe, GazeCloud), `EyeTrackerMock`
`IConsistencyProvider`	`load_document`, `add_highlight`, `get_annotations`, `search_text`, marcadores y comentarios	`ConsistencyPlugin`
`IWebSearchProvider`	`search_encyclopedia`, `search_definition`, `translate_text`, `search_image`, `search_video`	`LLMSearchPlugin` (Gemini)
`ISpeechProvider`	`speak`, `stop_speaking`, `start_listening`, `stop_listening`	`SpeechPlugin`

Fuente: Elaboración propia a partir de `contracts/`.

4.3 NÚCLEO ORQUESTADOR

4.3.1 Ciclo de vida

El `OrchestratorCore` lee `config/plugins.json`, superpone los ajustes del usuario, carga los plugins con el `PluginLoader` y los inicializa respetando sus dependencias (por ejemplo, el reconocimiento de texto depende de la cámara de texto). En cada ciclo llama a `execute()` de cada plugin, primero los productores y luego los consumidores de resultados, y mide la latencia de cada uno. Al cerrar, libera los plugins en orden inverso al de inicialización. El Algoritmo 4.1 resume el ciclo; el aislamiento de fallos está en las líneas marcadas con `try`.

Algoritmo 4.1: Ciclo de vida del orquestador.

Entrada: archivo `plugins.json`, ajustes del usuario
Salida : resultados por ciclo, métricas de latencia, traza

$C \leftarrow$ leer configuración y superponer ajustes;
foreach *entrada* $e \in C$ *habilitada* **do**
  $clase \leftarrow$ `DeviceFactory.resolve`(e) ; `// real o simulada`
  try cargar $clase$ con `importlib` **except** registrar `load_failed`;
end
foreach *plugin* p *cargado, en orden de dependencias* **do**
  inyectar sensor declarado si p es `ICameraFrameConsumer`;
  try p.`initialize`$(config_p)$ **except** registrar falla y continuar;
end
while *el sistema esté activo* **do**
  $R \leftarrow \emptyset$;
  foreach *plugin activo* p *(productores primero, consumidores al final)* **do**
    $t_0 \leftarrow$ `perf_counter`();
    if p *es* `ICycleResultConsumer` **then** p.`consume_cycle_results`(R);
    try $R[p] \leftarrow p$.`execute`$()$ **except** $R[p] \leftarrow$ error, registrar;
    registrar latencia $t - t_0$; advertir si supera 8,3 ms;
  end
end
foreach *plugin* p *en orden inverso* **do**
  try p.`release`$()$ **except** registrar;
end

Además del ciclo, el orquestador ofrece tres operaciones que necesita la interfaz:

`invoke(plugin, método, …)`, que ejecuta una operación bajo demanda (por ejemplo, reconocer el texto resaltado o buscar una definición) con el mismo aislamiento de fallos y midiendo su latencia; `restart_plugin`, que reinicia un plugin con una configuración nueva y reconecta a sus consumidores; y `stop_plugin`. Cada acción queda registrada en una traza JSONL que sirve de evidencia para la evaluación.

4.3.2 Configuración declarativa

Todos los plugins se declaran en un único archivo. Cada entrada indica la clase, la implementación simulada si es un dispositivo, las dependencias y la configuración:

```
{ "name": "ImageProcessor",
  "class": "plugins.image_processor_plugin.ImageProcessorPlugin",
  "enabled": true,
  "dependencies": {"camera_sensor": "TextCamera"},
  "config": {"ocr_language": "spa+eng", "rotate_180": true} }
```

Agregar un módulo consiste en escribir la clase y agregar una entrada; si la clase no existe o tiene un error, el plugin queda registrado como fallido y el resto del sistema arranca igual, que es exactamente el caso que en EDDIE-2023 detenía la aplicación (E50). Los ajustes del usuario (cámara elegida, PDF, eye tracker) se guardan en un archivo aparte, `config/user_settings.json`, con las mismas claves que usaba `Settings2` en C#, y se superponen al cargar sin modificar `plugins.json`.

4.3.3 Capa de abstracción de hardware

La `DeviceFactory` decide, para cada dispositivo que declara `simulated_class`, si se carga la implementación real o la simulada. El modo se elige con la variable de entorno `EDDIE_DEVICE_MODE` (`real` o `sim`) o por dispositivo en su configuración. Las implementaciones simuladas generan señales con verdad terreno conocida: `MockCameraSensor` entrega una página con texto resaltado, un lápiz o una mano en posiciones conocidas, y `EyeTrackerMock` una trayectoria de mirada. Esto permite ejecutar el sistema completo y las pruebas automatizadas sin el laboratorio (RNF-05) y, en una sesión real, distinguir una falla del sensor de una falla del software: si el sistema funciona en modo simulado y no en el real, el problema está en el dispositivo o su conexión.

4.3.4 Integración con la interfaz

`EddieRuntime` conecta el orquestador con la interfaz. Un temporizador de Qt ejecuta `execute_cycle()` a 30 Hz en el hilo de la interfaz, mientras que los sensores capturan en hilos

propios, de modo que un ciclo nunca espera a la cámara. Las acciones lentas (OCR, consulta al modelo de lenguaje, guardado del PDF, voz) se ejecutan con `invoke` en un conjunto de hilos de trabajo y su resultado vuelve a la interfaz por una señal. Este esquema reemplaza los hilos y ventanas de carga *ad hoc* de EDDIE-2023.

4.4 MÓDULOS INTEGRADOS

Reconocimiento de texto. `ImageProcessorPlugin` recibe la cámara de texto, localiza la zona resaltada, la recorta y la transcribe con Tesseract (`spa+eng`). También calcula precisión y recall contra una lista de palabras esperadas, la métrica de la propuesta para la verificación funcional.

Interacción gestual. Tres plugins implementan `IGesturePlugin`: lápiz de color, mano y mouse. Comparten la calibración de cuatro puntos, la selección de un fragmento y su captura, y una opción común para rotar la imagen 180° cuando la cámara está montada invertida (hallazgo H-01).

Rastreo ocular. `EyeTrackerPlugin` implementa los protocolos de GazePoint (Open Gaze API, TCP 4242), TheEyeTribe (TCP 6555) y GazeCloud (WebSocket 3000). `GazeSessionPlugin` registra la sesión de mirada en CSV con las columnas de EDDIE-2023 y el clic por permanencia.

Consistencia. `ConsistencyPlugin` mantiene el PDF abierto y sincroniza resaltados, comentarios, marcadores y figuras en ambos sentidos con PyMuPDF. Cada guardado escribe una copia completa y reemplaza el original (hallazgo H-02).

Búsqueda. `LLMSearchPlugin` implementa los cinco servicios de `IWebSearchProvider` con Gemini 2.5 Flash. La clave se lee de `config/secrets.json` o de la variable `GEMINI_API_KEY`, que no se versionan.

Voz. `SpeechPlugin` lee los resultados con `pyttsx3` y escucha los comandos con `SpeechRecognition`.

4.5 DISEÑO DE LA INTERFAZ

La interfaz de EDDIE-2023 se reconstruyó primero como réplica de los formularios WinForms (a partir de los archivos `.Designer.cs`) para no perder ningún control, y luego se rediseñó conservando todos los controles y flujos con el mismo nombre y comportamiento (RNF-06). Los criterios del rediseño provienen de la evaluación de Ibaceta Jaña (2023) y de la operación en la sala:

- **Sin amarillo fuera del resaltado.** La cámara detectaba el botón de resaltado de la proyección en lugar del texto (EFD2). La paleta usa tinta azul oscura (#1B2440) y un acento verde azulado (#0B7A87); el amarillo queda reservado para el resaltado, que es lo que el OCR busca.
- **Controles de la proyección en la franja superior.** TheEyeTribe no detectaba la parte inferior de la proyección (E49) y el reconocimiento de texto fallaba lejos de la cámara (EFD4), por lo que los botones que se activan con la mirada se ubican en la franja superior. La zona de lectura y las cruces de calibración conservan su posición original, porque los algoritmos de reconocimiento dependen de ellas.
- **Estado real de cada módulo.** La pantalla de inicio muestra los pasos para preparar la sesión y el estado de cada módulo según el orquestador (U1), y los mensajes son avisos no modales que no detienen la lectura (U3).
- **Calibración desde el sensor.** Los puntos de calibración gestual se copian de la lectura actual del sensor en lugar de escribirse a mano (U2).
- **Español en toda la interfaz** (E14).

Las Figuras 4.2 a 4.5 muestran la interfaz final; la comparación con EDDIE-2023 está en el Apéndice C.



Figura 4.2: Pantalla de inicio con los pasos de preparación y el estado de cada módulo.

Fuente: Elaboración propia (captura del avance P-07 con dispositivos simulados).

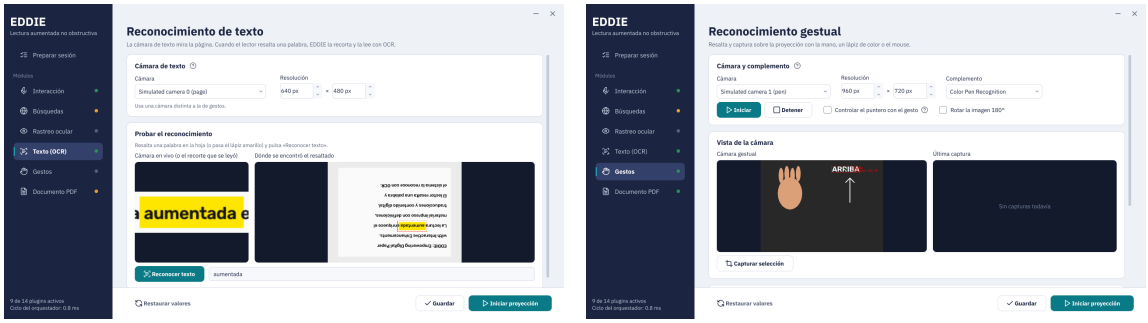


Figura 4.3: Configuración del reconocimiento de texto (izquierda) y de la interacción gestual con lápiz (derecha).

Fuente: Elaboración propia (capturas del avance P-07).

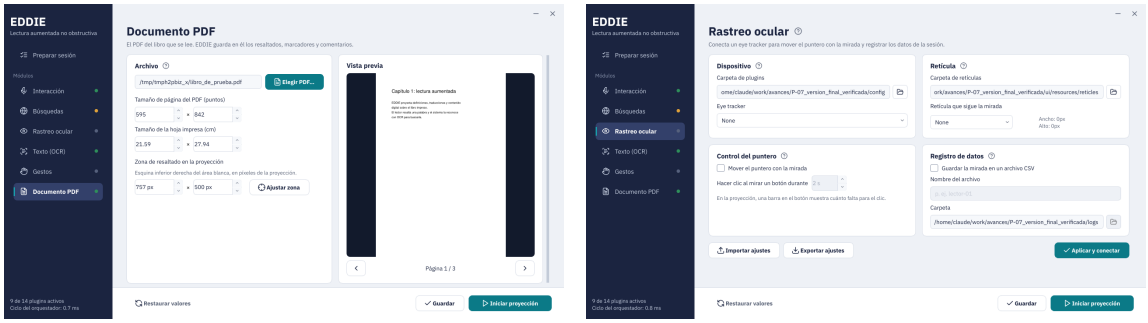


Figura 4.4: Configuración del documento PDF (izquierda) y del rastreo ocular (derecha).

Fuente: Elaboración propia (capturas del avance P-07).

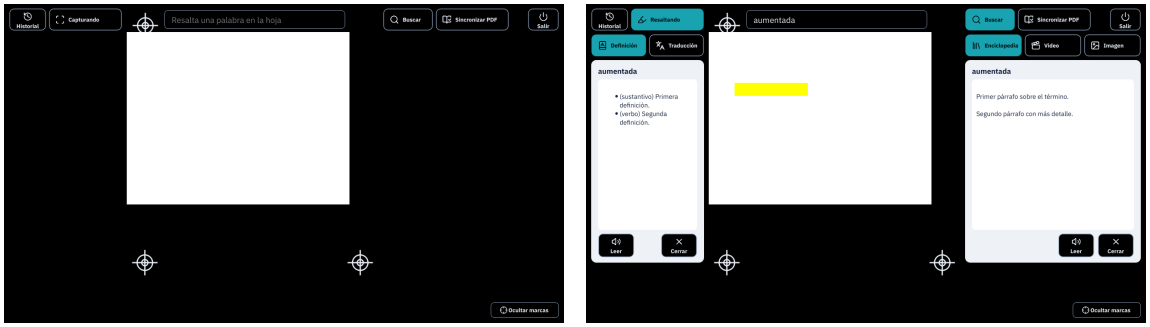


Figura 4.5: Pantalla proyectada: controles en la franja superior (izquierda) y resultado de una búsqueda (derecha).

Fuente: Elaboración propia (capturas del avance P-07 con un modelo de lenguaje de prueba).

4.6 ITERACIONES DE DESARROLLO

El sistema se construyó en ocho avances, cada uno con su código completo, pruebas y evidencias reproducibles (Tabla 4.2). P-00 corresponde a los prototipos desechables del Capítulo 3; P-01 a P-06 integran el núcleo y los módulos en el orden de menor a mayor dependencia del hardware; P-07 es la versión verificada. La interfaz se incorpora desde P-03 y muestra, en cada avance, solo los módulos ya integrados: los demás aparecen deshabilitados con la etiqueta del avance en que se integran. Las fichas de cada avance, con sus objetivos y requisitos, están en el Apéndice A.

Tabla 4.2: Avances del desarrollo y resultados de sus pruebas.

Avance	Contenido	PDCA	Pruebas	Fallos esperados	Cobertura
P-00	Prototipos desechables y mediciones	Plan	—	—	—
P-01	Núcleo orquestador y contratos	Do	34	0	81%
P-02	Capa de hardware y consistencia	Do	41	1	73%
P-03	Reconocimiento de texto e interfaz	Do	65	0	67%
P-04	Interacción gestual	Do	79	1	72%
P-05	Rastreo ocular	Do	94	1	74%
P-06	Proyección, búsqueda con LLM y voz	Do	100	2	81%
P-07	Versión final verificada	Check/Act	107	0	81%

Fuente: Elaboración propia a partir de `evidencias/` de cada avance.

Los fallos esperados (*xfail*) son pruebas que documentan un defecto conocido: en los avances en que el defecto existe la prueba se marca como fallo esperado, y en P-07, con la corrección aplicada, pasa a aprobada. Así cada corrección queda respaldada por una prueba que demuestra el problema y su solución (Sección 5.2.5).

CAPÍTULO 5. EVALUACIÓN

Este capítulo corresponde a la etapa *Check* del ciclo PDCA y al cuarto objetivo específico. La evaluación se organiza en los tres niveles del diseño experimental de la propuesta: pruebas de software, verificación y validación con MECIVA PS, y pruebas de aceptación en el laboratorio. El primer nivel se ejecutó completo con dispositivos simulados; los dos siguientes requieren el hardware del laboratorio, y para ellos se presentan el protocolo, los instrumentos ya construidos y los resultados que faltan, marcados como [Pendiente: laboratorio].

5.1 PLAN DE EVALUACIÓN

La Tabla 5.1 resume las métricas comprometidas, su criterio de aceptación y su estado. Todas las métricas automatizadas se regeneran con `python tools/generar_evidencias.py` en cada avance, y los resultados de este capítulo provienen de la carpeta `evidencias/` del avance P-07 (Linux 6.18.44-fc-v310, Python 3.13.18, 2026-11-08).

Tabla 5.1: Métricas de evaluación, criterio de aceptación y estado.

Métrica	Criterio	Estado
Nivel 1: pruebas de software		
Cobertura de líneas	$\geq$ 80% (RNF-03)	81%, cumple
Latencia orquestador $\rightarrow$ plugin	p95 $\leq$ 8,3 ms (RNF-01)	1,48 ms el ciclo completo, cumple
Aislamiento de fallos	Una falla no detiene el sistema (RNF-02)	Verificado por pruebas
Matriz de trazabilidad	Cada caso: corregido, persistente o fuera de alcance	Parcial (Sección 5.2.4)
Desfase entre sensores y pérdida de eventos	Desfase máximo (ms) y tasa de pérdida en captura simultánea	[Pendiente: laboratorio]
Precisión y recall del OCR	$\geq$ 10 palabras de distinto largo	Métrica implementada; [Pendiente: cámara]
Tasa de error gestual	Gestos bien clasificados / ejecutados	[Pendiente: laboratorio]
Nivel 2: verificación y validación (MECIVA PS)		
Requisitos críticos	100% de las ejecuciones	[Pendiente: laboratorio]
Validez funcional	% de requisitos heredados cumplidos	[Pendiente: laboratorio]
Confiabilidad	CV o tasa de éxito, $\geq$ 10 ejecuciones por caso	[Pendiente: laboratorio]
Nivel 3: pruebas de aceptación		
Requisitos secundarios	Escala A–E (Tabla 5.5)	[Pendiente: laboratorio]

Fuente: Elaboración propia a partir de la propuesta de tesis.

5.2 PRUEBAS DE SOFTWARE

5.2.1 Pruebas automatizadas y cobertura

La versión final tiene 107 pruebas automatizadas con pytest, organizadas en cinco archivos: núcleo y cargador (`test_orchestrator.py`), hardware y núcleo extendido (`test_hardware_and_core.py`), contratos y plugins (`test_plugins_eddie.py`), flujos de la interfaz (`test_ui_flows.py`) e interfaz rediseñada (`test_interfaz.py`). Las pruebas de interfaz ejecutan los flujos reales de la aplicación (abrir un PDF, sincronizar un resaltado, calibrar un gesto, abrir la proyección) con dispositivos simulados y sin pantalla. Todas aprueban y la cobertura de líneas es de 81%, sobre el umbral de 80% comprometido.

La Tabla 4.2 (Capítulo 4) muestra la evolución por avance. La cobertura de P-02 a P-05 está bajo el umbral porque esos avances contienen código de interfaz o de dispositivos que todavía no tenía pruebas de flujo completas; el umbral se exige a la versión final, que es la que se entrega.

5.2.2 Latencia del orquestador

Se midieron 500 ciclos completos del orquestador, tras 10 de calentamiento, con todos los plugins activos y el reconocimiento con lápiz encendido (Tabla 5.2). El ciclo completo, que incluye a todos los plugins, tiene una latencia de 0,59 ms en promedio y 1,48 ms en el percentil 95, con un máximo de 2,31 ms. Ningún plugin superó los 8,3 ms en ningún ciclo, por lo que RNF-01 se cumple con margen: el ciclo completo usa menos del 10% del período entre dos muestras del eye tracker.

Tabla 5.2: Latencia por plugin en 500 ciclos del orquestador (ms).

Plugin	**Media**	**p95**	**Máx.**
TextCamera	0,117	0,182	0,76
GestureCamera	0,264	0,369	2,12
ImageProcessor	0,006	0,008	0,057
ColorPenRecognition	0,122	0,932	1,189
HandSkinRecognition	0,003	0,005	0,046
MouseRecognition	0,002	0,003	0,013
EyeTrackerMock	0,01	0,015	0,16
SpeechProvider	0,003	0,004	0,079
GazeSession	0,005	0,006	0,13
ConsistencyProvider	0,009	0,011	0,074
Ciclo completo	0,59	1,48	2,31

Fuente: Elaboración propia, `evidencias/latencias.json` de P-07 (dispositivos simulados).

Dos limitaciones acotan este resultado. Primero, se midió con dispositivos simulados:

la latencia de captura de la cámara real y la del eye tracker ocurren en hilos propios y no forman parte del ciclo, pero el costo del reconocimiento sobre imágenes reales puede ser mayor. Segundo, en P-06 se registró un ciclo aislado de 10,28 ms con un percentil 95 de 1,49 ms; corresponde a una pausa del sistema operativo o del recolector de memoria y no se repitió en P-07, pero muestra que el máximo debe medirse también en el PC del laboratorio con `generar_evidencias.py --real`.

5.2.3 Aislamiento de fallos

El aislamiento de fallos (RNF-02) se verifica con pruebas que provocan fallas en cada fase del ciclo de vida: una clase que no existe o un módulo inválido (`test_load_invalid_module`, `test_load_all_missing_class_key`), un archivo de configuración ausente (`test_initialize_with_missing_config_file`), un plugin cuya inicialización falla (`test_initialize_plugin_init_fail_does_not_stop_system`), una excepción en `execute()` (`test_execute_plugin_exception_isolation`), una excepción al liberar (`test_release_exception_does_not_stop_others`) y una operación bajo demanda que falla (`test_invoke_isolates_failures`). En todos los casos el resto de los plugins sigue funcionando y la falla queda registrada con su causa.

El informe de inicialización de P-07 muestra el mismo comportamiento en la aplicación real: sin clave de Gemini configurada, el plugin de búsqueda queda como `initialization_failed` y los otros trece plugins se inicializan normalmente. En EDDIE-2023, una situación equivalente (una DLL o carpeta de plugins faltante, E50) impedía usar el módulo completo.

5.2.4 Matriz de trazabilidad

La propuesta compromete reejecutar los casos de prueba documentados por Pezoa Riquelme (2020), Contreras Velásquez (2019) y Estay Vera (2020a) en EDDIE-2023 y en la versión en Python, y registrar cada error de Ibaceta Jaña (2023) como corregido, persistente o fuera de alcance, con su evidencia. La Tabla 5.3 presenta la parte de la matriz que puede respaldarse con pruebas automatizadas; los casos que dependen del hardware quedan como pendientes hasta su ejecución en el laboratorio. La matriz completa de los 67 errores se construye con la plantilla del Anexo A.

Tabla 5.3: Matriz de trazabilidad (parcial) de errores de EDDIE-2023.

Error	Descripción	Cat.	Resultado	Evidencia
E50	Carpeta `PluginsConsistencia` no encontrada	(a)	Corregido por diseño	`test_load_invalid_module`, `test_load_all_missing_class_key`
E7, E8	No inicia sin cámara o sin dos cámaras	(d)	Corregido	`test_camera_sensor_fails_cleanly_when_camera_missing`; modo simulado
E11, E21, E22, E30	Fallas al sincronizar, buscar o revisar sin PDF o sin página	(d)	Corregido	`test_missing_pdf_is_reported_not_crashing`, `test_pdf_tools_require_pdf_and_page_number`
E12, E13	Sin manual de instalación ni de uso	(c)	Corregido	README del repositorio y manual del Apéndice D
E14	Mensajes en otro idioma	(e)	Corregido	Interfaz y mensajes en español (RNF-06)
EFD2	La cámara detecta botones de la interfaz	(e)	Corregido por diseño; [Pendiente: verificar con cámara]	`test_interface_chrome_never_uses_yellow`
E27, E28	Código sin comentarios ni nombres representativos	(d)	[Pendiente: SonarQube]	Docstrings y análisis estático
E40, E41	Resaltado digital no coincide con el físico	(d)	[Pendiente: laboratorio]	Prueba de aceptación de consistencia
E15, E16	Eye tracker no reconocido; cámara inestable	(b)	[Pendiente: laboratorio]	Modo real vs. simulado (DeviceFactory)
E49	No se detecta la parte inferior de la proyección	(b)	Mitigado por diseño; [Pendiente: laboratorio]	Controles en la franja superior; prueba con GazePoint
APIs de búsqueda	Credenciales vencidas	(g)	Corregido por reemplazo	`test_gemini_key_from_secrets_file_or_missing` y servidor de prueba
E9, E10	Conexiones eléctricas y seguridad del equipo	(f)	Fuera de alcance	Infraestructura del laboratorio

Fuente: Elaboración propia a partir de Ibaceta Jaña (2023) y de las pruebas de P-07. Categorías de la Tabla **??**: (a) carga y despliegue, (b) hardware, (c) documentación, (d) práctica de código, (e) interfaz e idioma, (f) infraestructura, (g) servicios externos.

5.2.5 Hallazgos de la verificación

Durante el desarrollo y la verificación se encontraron tres defectos que las pruebas automatizadas no habían detectado. Cada uno se documentó con una prueba que lo reproduce y se corrigió (Tabla 5.4).

Tabla 5.4: Hallazgos de la verificación y su corrección.

ID	Problema	Corrección y prueba
H-01	Al calibrar con la mano la imagen aparecía rotada 180°: el plugin heredó de `HandSkinRecognition.cs` una rotación fija pensada para la cámara invertida del laboratorio.	La rotación pasa a ser una opción común de los tres plugins gestuales, desactivada por defecto, con una casilla en la interfaz. Pruebas `test_gesture_preview_keeps_camera_orientation_by_default` y relacionadas.
H-02	Con PyMuPDF 2.1, desde el segundo guardado incremental el PDF quedaba con su tabla de referencias dañada y al reabrirlo se perdían anotaciones.	Cada guardado escribe una copia completa y reemplaza el original, con reintento si el archivo está bloqueado. Pruebas `test_successive_saves_keep_every_annotation_on_disk` y relacionadas.
H-03	Los botones de ayuda de Rastreo ocular lanzaban una excepción en cada clic porque la señal de Qt reemplazaba el argumento del *callback*.	La función ignora los argumentos de la señal. Prueba `test_info_icons_and_buttons_with_help_do_not_raise`.

Fuente: Elaboración propia.

Los tres hallazgos tienen un rasgo común: el programa parecía funcionar. En H-02 el documento en memoria era correcto y el daño solo aparecía al reabrir el archivo; en H-03 la acción del botón se ejecutaba y la excepción quedaba en el registro. Esto refuerza la necesidad de las pruebas de aceptación con el hardware real, porque hay defectos que solo se observan al operar el sistema completo.

5.3 VERIFICACIÓN Y VALIDACIÓN CON MECIVA PS

5.3.1 Protocolo

La verificación replica la fase de diagnóstico de MECIVA PS (Ibaceta Jaña, 2023) para cada módulo del MVP: consistencia, reconocimiento de texto, interacción gestual y rastreo ocular. Para cada módulo:

1. Se clasifican sus requisitos en **críticos**, que deben cumplirse en el 100% de las ejecuciones y cuyo incumplimiento es bloqueante (por ejemplo, que los plugins del MVP carguen sin fallas, o que el error espacial del rastreo ocular esté dentro del margen definido), y **secundarios**, que se evalúan con la escala de la Tabla 5.5.

2. Se ejecuta cada caso al menos 10 veces con el hardware real (`EDDIE_DEVICE_MODE=real`) y se registra el resultado en la plantilla de diagnóstico de MECIVA PS (funcionalidad, resultado esperado, método, pruebas, resultados y nivel de funcionamiento).

3. Se calcula la **validez funcional** como el porcentaje de requisitos heredados que el módulo cumple, y la **confiabilidad** como el coeficiente de variación CV $= \sigma/\mu$ para variables

continuas (por ejemplo, las coordenadas de una fijación sobre un punto fijo) o como la tasa de éxito para resultados binarios (por ejemplo, que un comentario se sincronice).

Para el reconocimiento de texto se usa el mismo conjunto de Ibaceta Jaña (2023): diez palabras de distinto largo (Control, Arte, Lectura, Jerusalén, Esponja, Pastilla, Caramelo, Metodología, Consistencia y Cama) en nueve posiciones de la zona de lectura, con diez repeticiones por posición. Así los resultados son directamente comparables con la línea base de 2023, en que la validez varió entre 10% y 100% según la posición. El plugin calcula precisión y recall con `get_precision_recall` (verificado en `test_precision_recall_metric`):

$$\text{Precisión} = \frac{|P_{\text{reconocidas}} \cap P_{\text{esperadas}}|}{|P_{\text{reconocidas}}|}, \qquad \text{Recall} = \frac{|P_{\text{reconocidas}} \cap P_{\text{esperadas}}|}{|P_{\text{esperadas}}|}. \tag{5.1}$$

5.3.2 Resultados

[Pendiente: Tabla de validez y confiabilidad por módulo, con la línea base de Ibaceta Jaña (2023) en una columna para comparar.]

5.4 PRUEBAS DE ACEPTACIÓN

Las pruebas de aceptación se realizan en la sala del laboratorio InTeractiOn con el profesor guía como evaluador, siguiendo la práctica de los trabajos anteriores del laboratorio (Estay Vera, 2020a; Pezoa Riquelme, 2020; Figueroa Contreras, 2025). Cada prueba tiene precondiciones, pasos, requisitos abordados y resultado esperado, y se califica con la escala de la Tabla 5.5. Los escenarios cubren la operación simultánea de módulos, la recuperación ante la falla parcial de un sensor (desconectar una cámara durante la sesión) y la sincronización de datos entre componentes de frecuencias distintas.

Tabla 5.5: Escala de evaluación de las pruebas de aceptación.

Evaluación	Significado	Cumplimiento
A	Completa	85–100%
B	Aceptada con observaciones	55–84,9%
C	Incompleta	45–54,9%
D	Insuficiente	15–44,9%
E	No aceptada	0–14,9%

Fuente: Estay Vera (2020a).

Tabla 5.6: Pruebas de aceptación planificadas.

ID	Prueba	Requisitos	Resultado
PA-01	Arrancar con todos los plugins del MVP, desconectar una cámara durante la sesión y reconectarla desde la interfaz.	N1, N2, N6, RNF-02	[Pendiente:]
PA-02	Resaltar una palabra en el papel y obtener su texto, su definición y su traducción.	FD1, B1–B13	[Pendiente:]
PA-03	Sincronizar resaltado, comentario, marcador y figura del papel al PDF y del PDF al papel.	C1–C12	[Pendiente:]
PA-04	Calibrar el lápiz y la mano, seleccionar un fragmento y capturarlo.	G1–G3, U2	[Pendiente:]
PA-05	Conectar GazePoint y TheEyeTribe, ver la retícula y activar un botón de la proyección con la mirada.	RO1–RO7	[Pendiente:]
PA-06	Registrar una sesión de lectura y verificar el CSV de mirada y la traza del orquestador.	RO4, N7	[Pendiente:]
PA-07	Preparar una sesión completa desde la pantalla de inicio siguiendo el manual.	U1, U3, RFS1–RFS8	[Pendiente:]

Fuente: Elaboración propia.

5.5 SÍNTESIS DE LA EVALUACIÓN

Las pruebas de software muestran que la arquitectura cumple los requisitos no funcionales que pueden verificarse sin el laboratorio: cobertura de 81% (RNF-03), latencia del ciclo completo de 1,48 ms en el percentil 95 (RNF-01), aislamiento de fallos en todas las fases del ciclo de vida (RNF-02), extensibilidad por configuración (RNF-04) y ejecución completa con dispositivos simulados (RNF-05). De los errores de EDDIE-2023, los de carga y despliegue, práctica de código, documentación e idioma que se pudieron reproducir quedan corregidos y respaldados por pruebas. Los errores de hardware y la coincidencia física del resaltado (E40, E41, E49) solo pueden cerrarse con las pruebas del laboratorio, para las que el protocolo y los instrumentos de medición ya están disponibles.

CAPÍTULO 6. CONCLUSIONES

Versión preliminar: las conclusiones sobre el cuarto objetivo se completan con los resultados de la verificación funcional y de las pruebas de aceptación en el laboratorio.

6.1 CUMPLIMIENTO DE LOS OBJETIVOS

OE1: diagnóstico y factibilidad. La reclasificación por causa de los 67 errores de Ibaceta Jaña (2023) mostró que 42 de ellos (62,7%) dependen de la arquitectura (carga y despliegue, y práctica de código) y no del hardware, lo que justifica reconstruir el núcleo en lugar de corregir errores aislados. Los 19 prototipos desechables confirmaron que las funciones críticas pueden reproducirse en Python con bibliotecas mantenidas y, al medirse con verdad terreno conocida, entregaron cinco hallazgos que no eran visibles en el código del sistema legado: la binarización global falla con la luz del proyector, el preprocesamiento empeora el OCR sobre texto limpio, el contenido proyectado del color del lápiz anula su seguimiento, el orden de la regla de clasificación de figuras confunde cuadrados con círculos y el clic por permanencia depende fuertemente del ruido del sensor. El objetivo se considera cumplido, con las mediciones que dependen del hardware (F1, F9, F24, F25 y la parte de sensor de las demás) pendientes de ejecutar en el laboratorio con el mismo script.

OE2: diseño del núcleo. Se definió un contrato base único (`PluginBase`) con ciclo de inicialización, ejecución y liberación, siete contratos de dominio y dos interfaces de flujo de datos, un único archivo de configuración y una capa de abstracción de hardware con conmutación real/simulado. El diseño reemplaza los tres mecanismos de carga de EDDIE-2023 por uno solo. El objetivo se considera cumplido.

OE3: implementación e integración. Se implementó el núcleo y se integraron, en siete iteraciones, los módulos de consistencia, reconocimiento de texto, interacción gestual (lápiz, mano y mouse), rastreo ocular (GazePoint, TheEyeTribe, GazeCloud y simulado), búsqueda con modelo de lenguaje y voz, junto con la interfaz de configuración y la pantalla proyectada. La interfaz conserva todos los controles y flujos de EDDIE-2023 y corrige los problemas de usabilidad registrados en 2023. El objetivo se considera cumplido en software; la integración con el hardware físico se confirma en las pruebas de aceptación.

OE4: validación. Las pruebas de software se completaron: 107 pruebas aprobadas, 81% de cobertura de líneas, latencia del ciclo completo de 1,48 ms en el percentil 95 frente al umbral de 8,3 ms y aislamiento de fallos verificado en todas las fases del ciclo de vida. La verificación funcional con MECIVA PS y las pruebas de aceptación [Pendiente: se ejecutan en el laboratorio]; sus protocolos e instrumentos están listos (Capítulo 5).

6.2 RESPUESTA A LA PREGUNTA DE INVESTIGACIÓN

La pregunta planteada fue en qué medida la reconstrucción del núcleo bajo una arquitectura de plugins con contrato único permite reducir los errores estructurales documentados y habilitar el uso de EDDIE como instrumento de medición confiable. Respecto de la primera parte, los errores de carga y despliegue dejan de ser posibles por construcción: un plugin faltante o defectuoso se registra y el resto del sistema arranca, y los errores de práctica de código que se reprodujeron quedan respaldados por pruebas. Respecto de la segunda, la arquitectura entrega las condiciones necesarias para medir (latencia acotada, traza de cada acción, ejecución reproducible con dispositivos simulados), pero la confiabilidad de las mediciones con el hardware real solo puede afirmarse después de la verificación con MECIVA PS.

6.3 LIMITACIONES

- Los resultados de los prototipos marcados como sintéticos miden el algoritmo sobre datos con verdad terreno conocida, no el desempeño con la cámara o el eye tracker reales.
- Las pruebas de software se ejecutaron con dispositivos simulados; la latencia y el comportamiento con el hardware real deben medirse en el PC del laboratorio.
- La búsqueda depende de un servicio externo (Gemini) y de una clave personal; si el servicio cambia sus condiciones, el contrato `IWebSearchProvider` permite reemplazarlo, pero el reemplazo no está implementado.
- El desarrollo y la validación se realizan solo en Windows 11.

6.4 TRABAJO FUTURO

- Ejecutar la verificación con MECIVA PS y las pruebas de aceptación, y completar la matriz de trazabilidad de los 67 errores.
- Calibrar los parámetros del clic por permanencia y los umbrales de color con el hardware del laboratorio.
- Implementar un proveedor de búsqueda local (por ejemplo, un modelo de lenguaje ejecutado en el equipo) para no depender de un servicio externo.
- Retomar la detección automática del libro (F4), que resultó factible, y el canal de telemetría para el visor web que quedó fuera del alcance.

GLOSARIO

Arquitectura modular: Sistema de construcción que utiliza componentes prefabricados (módulos), que luego se ensamblan en el sitio final.

Capa de abstracción de hardware: Componente de software que actúa como intermediario entre el sistema operativo o aplicación y el hardware físico, permitiendo interactuar con distintos dispositivos mediante una interfaz uniforme sin conocer sus detalles técnicos específicos.

Consistencia bidireccional: Mecanismo de validación utilizado en diversos campos técnicos para asegurar que un proceso, dato o relación sea coherente tanto al analizarlo en una dirección como en la contraria.

Deuda técnica: Costo futuro de retrabajo derivado de elegir soluciones rápidas y fáciles hoy, en lugar de un enfoque mejor diseñado pero más lento.

Eyetrackers: Dispositivos tecnológicos que miden dónde mira una persona, la dirección de su mirada y el movimiento de sus ojos en tiempo real.

Interacción humano-información (HII): Campo de investigación que estudia cómo las personas buscan, acceden, evalúan y usan información en distintos contextos y formatos.

MECIVA PS: Metodología específica utilizada para la evaluación y validación de software científico, la cual permitió identificar los 67 errores estructurales en el sistema EDDIE actual.

Microservicios: Arquitectura de software donde el sistema se divide en servicios independientes que se comunican mediante APIs, evaluada como una alternativa de solución para EDDIE por su alta escalabilidad.

Orquestador: Sistema o herramienta que organiza, coordina y dispone diversos elementos para lograr un objetivo común, asegurando que funcionen de manera armoniosa.

Overhead de comunicación interproceso (IPC): Tiempo y recursos adicionales de procesamiento que el sistema consume para permitir que módulos independientes intercambien información entre sí.

Plugins: Aplicación informática pequeña que se integra en otro programa principal (como un navegador o gestor de contenido) para añadir funcionalidades específicas o mejorar su rendimiento sin modificar el código base.

Realidad aumentada: Tecnología que superpone información digital, como imágenes, texto o datos, sobre el entorno físico real percibido por el usuario, sin reemplazarlo.

Science gateways: Plataformas o portales que facilitan el acceso a recursos compartidos para la investigación científica, cuyas arquitecturas modulares sirven como referencia para la sostenibilidad de EDDIE.

SonarQube: Plataforma de análisis estático de código diseñada para evaluar y mejorar continuamente la calidad, seguridad y mantenibilidad del software.

Visión computacional: Área de la informática enfocada en permitir que las computadoras identifiquen y procesen objetos en imágenes o videos, utilizada en el proyecto para el reconocimiento de texto y gestos.

REFERENCIAS BIBLIOGRÁFICAS

Bustio Martínez, L., Coma Peña, Y., & Talavera Bustamante, I. (2013). Arquitectura basada en plugins para el desarrollo de software científico. In *XV Convención y Feria Internacional Informática*. La Habana, Cuba.

Carrasco Madriaga, O. I. (2017). *Diseño e implementación de un sistema de realidad aumentada para el apoyo de la lectura en papel*. Memoria de título, Universidad de Santiago de Chile.

Contreras Velásquez, C. E. (2019). *Desarrollo de un sistema para soportar lectura aumentada no obstrusiva*. Memoria de título, Universidad de Santiago de Chile.

Ernst, N. A., Bavota, G., & Cai, Y. (2026). The nature of technical debt in research software. Preprint. arXiv.
URL `https://arxiv.org/abs/2603.2041`

Estay Vera, G. H. (2020a). *Módulo de interacciones basadas en movimientos oculares para ambiente de lectura aumentada no obstrusiva*. Trabajo de titulación, Universidad de Santiago de Chile.

Estay Vera, J. S. (2020b). *Integración de hardware de eye-tracking al sistema EDDIE*. Memoria de título, Universidad de Santiago de Chile.

Figueroa Contreras, M. J. (2025). *Arquitectura basada en plugins para la captura y análisis de datos multimodales: El caso de Multimodal Observer*. Memoria de título, Universidad de Santiago de Chile.

Glatard, T., Rousseau, M. E., Camarasu-Pop, S., Adalat, R., Beck, N., Das, S., Ferreira da Silva, R., Khalili-Mahani, N., Korkhov, V., Quirion, P. O., et al. (2017). Software architectures to integrate workflow engines in science gateways. *Future Generation Computer Systems*, *75*, 239–255.

Ibaceta Jaña, J. M. (2023). *Evaluación y validación de software científico mediante metodología MECIVA PS: Caso de estudio EDDIE*. Memoria de título, Universidad de Santiago de Chile.

Montanares Mundaca, D. C. (2019). *Desarrollo de un módulo de búsqueda web para el software EDDIE*. Memoria de título, Universidad de Santiago de Chile.

Orozco Solano, A. (2017). *Diseño de Sistemas Bioinformáticos y Biología Computacional aplicados en Nanobiotecnología Molecular Médica*. Tesis doctoral, Universidad Autónoma de Madrid.

Pezoa Riquelme, D. E. (2020). *Implementación de un módulo de consistencia bidireccional físico-digital para el sistema EDDIE*. Memoria de título, Universidad de Santiago de Chile.

Santibáñez Espinoza, F. (2020). *Rediseño de interfaz de usuario para el sistema EDDIE bajo un enfoque centrado en el usuario*. Memoria de título, Universidad de Santiago de Chile.

ANEXO A. ERRORES DE EDDIE-2023 Y PLANTILLA DE TRAZABILIDAD

La Tabla A.1 reproduce un extracto de los errores registrados por Ibaceta Jaña (2023) que se citan en este documento. La matriz completa se construye con la plantilla de la Tabla A.2, una fila por cada uno de los 67 errores, ejecutando el caso de prueba original en EDDIE-2023 y en la versión en Python.

Tabla A.1: Extracto de errores encontrados en EDDIE-2023.

ID	Tipo	Descripción	Estado
E7	Interno-otro	El programa no se puede inicializar por falta de cámara disponible.	SF
E8	Interno-otro	El programa no funciona correctamente sin dos cámaras disponibles.	SF
E9	Externo	Falta de conexiones eléctricas en el laboratorio.	SP
E10	Externo	Falta de seguridad en el computador del laboratorio.	SF
E11	Interno-consistencia	La sincronización hace fallar el programa cuando no se ingresa un PDF.	SF
E12	Externo	No existe un manual de instalación.	SS
E13	Externo	No existe un manual de uso.	SS
E14	Interno-otro	Los mensajes de error están en un idioma distinto al resto de la aplicación.	SF
E15	Externo	No se reconoce el eye tracker en el computador.	SF
E16	Externo	La conexión de la cámara digital es inestable.	SP
E21	Interno-consistencia	El botón de búsqueda hace fallar el programa si no se sube un PDF.	SF
E22	Interno-consistencia	La revisión del contenido del PDF falla si no existe el archivo.	SF
E27	Interno-otro	Faltan comentarios en el código.	SS
E28	Interno-otro	Variables y funciones sin nombres representativos.	SS
E30	Interno-consistencia	El contenido del PDF falla si no se indica una página.	SF
E48	Interno-consistencia	La consistencia de comentarios falla con datos correctos.	SF
E49	Externo	Con TheEyeTribe no se detecta la parte inferior de la proyección.	SS
E50	Interno-consistencia	No se encuentra la carpeta `PluginsConsistencia`.	SF

Fuente: Ibaceta Jaña (2023). SF: solución final; SP: solución parcial; SS: sin solución.

Tabla A.2: Plantilla de la matriz de trazabilidad.

ID	Módulo	Estado C#	Cat.	¿Reproducido?	Resultado en Python	Evidencia
E*n*	…	SF/SP/SS	(a)–(g)	Sí/No	Corregido / Persiste / Fuera de alcance	Prueba, log o nota de alcance

Fuente: Propuesta de tesis (Tabla 3.3).

ANEXO B. PLAN DE TRABAJO

Tabla B.1: Plan de trabajo.

Semanas	Fase	Actividades	OE	Horas	Entregable
1–3	Plan	Diagnóstico de la arquitectura en C# mediante la reclasificación de errores y prototipos desechables de factibilidad.	OE1	108	Diagnóstico y resultados de prototipos
4–6	Do (diseño)	Diseño del núcleo y de los contratos de interfaz; primer prototipo funcional.	OE2	108	Arquitectura y contratos
7–10	Do (implem.)	Núcleo, capa de abstracción de sensores, consistencia, reconocimiento de texto e interacción gestual.	OE3	144	Prototipos funcionales del núcleo con módulos
11–12	Do (implem.)	Rastreo ocular con hardware físico; contingencia: validación con `EyeTrackerMock`.	OE3	72	Prototipo integrado
13–14	Check	Pruebas de software: cobertura, trazabilidad, latencia y métricas de sincronización y visión.	OE4	72	Reporte de pruebas de software
15–16	Check	MECIVA PS por módulo, validez y confiabilidad, pruebas de aceptación.	OE4	72	Reporte de verificación y validación
17	Act	Ajustes finales, cierre de la documentación y entrega.	OE4	36	Documentación y entrega final

Fuente: Propuesta de tesis (Tabla 6.1).

APÉNDICE A. FICHAS DE LOS AVANCES

Cada avance se describe con el formato de Figueroa Contreras (2025): identificador, nombre, objetivos, descripción y requisitos funcionales y no funcionales. Las tablas se generan desde la carpeta `avances/` del repositorio.

Tabla A.1: Primer prototipo - Núcleo orquestador y contratos de plugins.

ID Prototipo	P-01
Nombre	Núcleo orquestador y contratos de plugins
Objetivos	Definir PluginBase y los contratos de cada módulo (ICameraSensor, IImageProcessor, IGesturePlugin, IEyeTracker, IConsistencyProvider, IWebSearchProvider, ISpeechProvider). Implementar el OrchestratorCore (inicializar $\rightarrow$ ejecutar $\rightarrow$ liberar) y el PluginLoader con un único mecanismo de carga. Validar la inyección de dependencias sensor$\rightarrow$consumidor, el aislamiento de fallos y la latencia por ciclo.
Descripción	Primera versión del núcleo, sin interfaz. Se prueba con dispositivos simulados (MockCameraSensor, EyeTrackerMock) y con plugins de prueba: un consumidor de frames, un consumidor de resultados del ciclo, un plugin que falla cada 10 ciclos y una entrada con una clase inexistente (equivalente a la DLL faltante de los errores E38/E50). El OrchestratorCore de este avance es la versión temprana del repositorio (sin overrides, invoke ni reinicio en caliente).
Requisitos Funcionales	N1, N2, N3, RFCO12
Requisitos No Funcionales	RNF-01, RNF-02, RNF-04, RNF-05

Fuente: Elaboración propia, 2026.

Tabla A.2: Segundo prototipo - Capa de abstracción de hardware y módulo de consistencia.

ID Prototipo	P-02
Nombre	Capa de abstracción de hardware y módulo de consistencia
Objetivos	Incorporar DeviceFactory para conmutar cada dispositivo entre real y simulado sin cambiar código. Implementar la cámara real (OpenCV, hilo de captura) y la enumeración de cámaras de Windows. Integrar primero el módulo sin dependencia de hardware: la consistencia papel$\leftrightarrow$PDF como plugin IConsistencyProvider (PyMuPDF). Extender el núcleo con overrides de configuración, invocación bajo demanda, reinicio en caliente y traza de ejecución.
Descripción	El núcleo pasa a su versión definitiva y se agrega la capa HAL. La consistencia (resaltados, marcadores, comentarios y figuras, búsqueda de texto) se ejecuta a través del orquestador con OrchestratorCore.invoke(). Sigue sin interfaz: se demuestra con demos/demo_consistencia.py, que sincroniza tres «fotos» sintéticas de la hoja con un PDF de prueba y guarda el PDF resultante y el render de cada página.
Requisitos Funcionales	N4, N5, N6, N8, C1, C2, C3, C7, C9, C10, C11 (parcial)
Requisitos No Funcionales	RNF-01, RNF-02, RNF-04, RNF-05

Fuente: Elaboración propia, 2026.

Tabla A.3: Tercer prototipo - Reconocimiento de texto e interfaz de configuración.

ID Prototipo	P-03
Nombre	Reconocimiento de texto e interfaz de configuración
Objetivos	Integrar el reconocimiento del texto resaltado (IImageProcessor: segmentación del resaltado + Tesseract) sobre la cámara de texto declarada. Construir la ventana de configuración a partir de los controles extraídos de los .Designer.cs, con un diseño propio en español (ui/theme.py, ui/modern_forms.py) y las páginas Texto (OCR) y Documento PDF. Ejecutar el ciclo del orquestador dentro de Qt (EddieRuntime) y persistir los ajustes. Poder probar el OCR y ver el PDF cargado desde estas páginas, sin esperar la pantalla de proyección.
Descripción	Aparece la interfaz. Conserva todos los controles del original (mismo nombre, tipo, rangos y estado inicial), ordenados en tarjetas con layouts de Qt, con un menú lateral que muestra el estado de cada módulo y una pantalla de inicio «Preparar sesión». Solo se construyen las páginas de los módulos ya integrados: las demás aparecen en el menú deshabilitadas con el avance en que se integran, y «Iniciar proyección» avisa que la proyección llega en P-06. Como en el C# el texto reconocido solo se veía desde la proyección, se agregan el botón «Reconocer texto» con el texto reconocido y una vista previa del PDF con navegación de páginas. Los mensajes son avisos no modales.
Requisitos Funcionales	FD1, N7, U1, U3, RFS3, RFS5, RFS6, RFS7, RFS8
Requisitos No Funcionales	RNF-01, RNF-06, RNF-07

Fuente: Elaboración propia, 2026.

Tabla A.4: Cuarto prototipo - Interacción gestual.

ID Prototipo	P-04
Nombre	Interacción gestual
Objetivos	Integrar los plugins gestuales del C# (Color Pen, Hand Skin, Mouse) bajo IGesturePlugin, compartiendo la cámara gestual declarada. Portar la calibración sensor→aplicación, el control del cursor («Recognition on»), la máquina de estados del clic y la captura automática de la selección. Agregar la página Gestos, con calibración asistida («Fijar inicio» / «Fijar fin» desde la lectura del sensor).
Descripción	Los tres detectores heredan una base común (CalibratedGesture-Plugin) que aplica la calibración y el clic; cada uno solo implementa su detección. Este avance porta el comportamiento del C# tal cual, incluida la rotación fija de 180° de Hand Skin, que después se detectó como defecto (H-01).
Requisitos Funcionales	G1, G2, G3, U2
Requisitos No Funcionales	RNF-01, RNF-04, RNF-06

Fuente: Elaboración propia, 2026.

Tabla A.5: Quinto prototipo - Rastreo ocular.

ID Prototipo	P-05
Nombre	Rastreo ocular
Objetivos	Unificar GazePoint (Open Gaze API), TheEyeTribe y GazeCloud bajo IEyeTracker, con EyeTrackerMock como contingencia (DeviceFactory). Portar la sesión de mirada: control del mouse por mirada, clic por permanencia, retícula y registro CSV. Agregar la página Rastreo ocular (exportar/importar configuración).
Descripción	Los tres protocolos se prueban con servidores falsos que emiten el mismo formato que los dispositivos reales. La sesión de mirada es un consumidor de resultados del ciclo: recibe la muestra del eye tracker en el mismo ciclo en que se produce.
Requisitos Funcionales	RO1–RO7, RFS4
Requisitos No Funcionales	RNF-01, RNF-02, RNF-05

Fuente: Elaboración propia, 2026.

Tabla A.6: Sexto prototipo - Pantalla de proyección, búsqueda con LLM y voz.

ID Prototipo	P-06
Nombre	Pantalla de proyección, búsqueda con LLM y voz
Objetivos	Integrar la pantalla de proyección (ProjectionScreenActivity2): modo resaltado/captura, búsqueda y sincronización papel↔PDF, con los botones en la franja superior (zona de mayor validez del eye tracker según Ibaceta, 2023) y la zona de lectura y las cruces en su posición original. Responder los cinco servicios de búsqueda con Gemini 2.5 Flash bajo IWebSearchProvider, en lugar de las APIs vencidas (Wikipedia, Google, Bing, YouTube, Cloud Vision). Integrar lectura en voz alta y comandos de voz (ISpeechProvider) y las páginas Interacción y Búsquedas.
Descripción	Con este avance el sistema queda completo y equivale al código de «Proyecto final» antes de la verificación. Es una extensión respecto del alcance del seminario, donde IWebSearchProvider quedaba solo como interfaz.
Requisitos Funcionales	B1–B13, C5, C6, C8, C11, C12, RFS1, RFS2
Requisitos No Funcionales	RNF-01, RNF-06, RNF-08

Fuente: Elaboración propia, 2026.

Tabla A.7: Séptimo prototipo - Versión final verificada.

ID Prototipo	P-07
Nombre	Versión final verificada
Objetivos	Verificar el sistema integrado: cobertura, latencia, trazabilidad de errores y pruebas de regresión. Corregir los hallazgos de la verificación (H-01, H-02, H-03) y dejar una prueba que los detecte. Cerrar la documentación técnica (README, EQUIVALENCIAS, DISENO_INTERFAZ).
Descripción	Es el contenido de «Proyecto final» con las correcciones de la verificación. Cada hallazgo tiene una prueba de regresión que falla con el código de P-06 y pasa con el de P-07.
Requisitos Funcionales	Todos los anteriores
Requisitos No Funcionales	RNF-01, RNF-02, RNF-03, RNF-04, RNF-05, RNF-06, RNF-07, RNF-08

Fuente: Elaboración propia, 2026.

Tabla A.8: Hallazgos de la verificación.

ID	Descripción	Estado	Justificación o Solución
H-01	Al calibrar con Hand Skin la vista de la cámara aparece rotada 180°. HandSkinRecognition.cs rotaba siempre la imagen porque la cámara del laboratorio estaba montada invertida; el port conservó esa rotación fija, solo en ese plug-in (Color Pen y Mouse no rotan), así que la vista previa y las coordenadas de calibración quedaban al revés con cualquier otra cámara.	SF	La rotación pasa a ser una opción común de los tres plug-ins (rotate_180, por defecto desactivada) que se aplica una sola vez al frame; vista previa, coordenadas y recorte comparten orientación. Casilla nueva «Rotar la imagen 180°» en Gestos para el montaje invertido del laboratorio.
H-02	ConsistencyPlugin guardaba cada cambio con saveIncr(). Con PyMuPDF 2.1, desde el segundo guardado incremental el PDF queda con la xref dañada; al reabrirlo PyMuPDF lo «repara» y se pierden los resaltados, comentarios y marcadores guardados antes. En pantalla no se notaba porque el documento en memoria seguía correcto.	SF	Cada guardado escribe una copia completa en un archivo temporal, reemplaza el original y vuelve a abrirlo (se mantiene el respaldo .bac). Si otro programa tiene el PDF bloqueado (Windows), se reintenta y luego se informa con un mensaje claro.
H-03	Al pulsar los botones de carpeta o las casillas de Rastreo ocular aparecía «'bool' object has no attribute 'mapToGlobal'». La ayuda contextual se conectaba con lambda w=widget, t=text a la señal clicked; en los botones, clicked(bool) entrega checked y Qt lo pasaba como primer argumento, reemplazando w por False. La acción del botón sí se ejecutaba, pero cada clic dejaba una excepción.	SF	La ayuda por clic queda solo para los íconos de información; los botones y casillas la muestran al pasar el mouse. La lambda ignora los argumentos de la señal (lambda *_, w=widget, t=text). Se corrige desde P-05, primer avance con esos controles, hasta P-07.

Fuente: Elaboración propia, 2026.

APÉNDICE B. MEDICIONES DE LOS PROTOTIPOS

La Tabla B.1 resume el tipo de medición y el resultado principal de cada prototipo. El detalle (protocolo, fórmula, parámetros y todas las métricas) está en `avances/P-00_prototipos_desechables/evidencias/MEDICIONES.md`, generado junto con los valores de este documento.

Tabla B.1: Resumen de las mediciones de los prototipos.

ID	Tipo	Resultado principal
F1	laboratorio	[Pendiente: FPS y latencia con las cámaras]
F2	sintético	Eficacia Otsu 25,8%; adaptativo 96,8%
F3	pdf	CER 1,46% / WER 6,86% (sin preproceso: 0,42% / 0,71%)
F4	sintético	Éxito 97,5%, RMSE 2,26 px
F5	sintético	Detección 100% sin rojo proyectado; 0% con rojo
F6	sintético	IoU 96,5%, punta correcta 96,5%
F7	sintético	Error de reproyección 1,53 $\pm$ 0,51 px
F9	laboratorio	[Pendiente: latencia de eventos]
F10	sintético	Pérdida 0%, *jitter* 0,45 ms
F11	código	0 fallas en 240 cargas, 0,36 ms
F12	sintético	100% voluntarias, 3,5 accidentales/min
F13	sintético	0,82 ms por cuadro
F14	pdf	Coherencia 82%
F15	pdf	48/48 persistidos, 13,8 ms
F16	sintético	100% detectadas, 0,68 pt
F17	sintético	Exactitud 80%, IoU 90,1%
F24	laboratorio	[Pendiente: exactitud de comandos]
F25	laboratorio	[Pendiente: palabras por minuto]
F27	sintético	0,58 ms por cuadro

Fuente: Elaboración propia, `mediciones/medir_prototipos.py` (P-00), 2026-11-08.

Para repetir las mediciones en el PC y agregar las de hardware:

```
cd avances\P-00_prototipos_desechables
python mediciones\medir_prototipos.py --pdf ruta\al\documento.pdf --camara 0
python ..\..\tesis\datos\actualizar_datos.py ..
```

El segundo comando copia los valores nuevos a `datos/`, y al recompilar el documento todas las tablas y fichas se actualizan.

APÉNDICE C. COMPARACIÓN DE LA INTERFAZ

Las figuras siguientes comparan la interfaz de EDDIE-2023, reproducida a partir de sus archivos `.Designer.cs`, con la versión en Python.

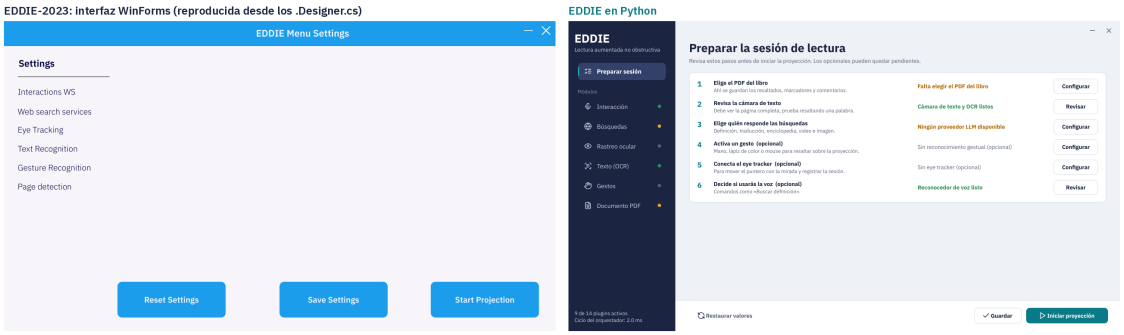


Figura C.1: Pantalla de inicio: EDDIE-2023 (izquierda) y versión en Python (derecha).

Fuente: Elaboración propia.

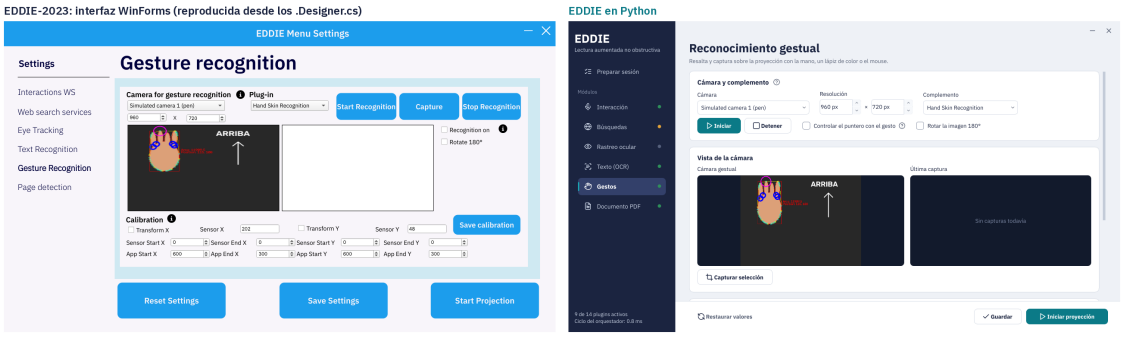


Figura C.2: Configuración de gestos: EDDIE-2023 (izquierda) y versión en Python (derecha).

Fuente: Elaboración propia.

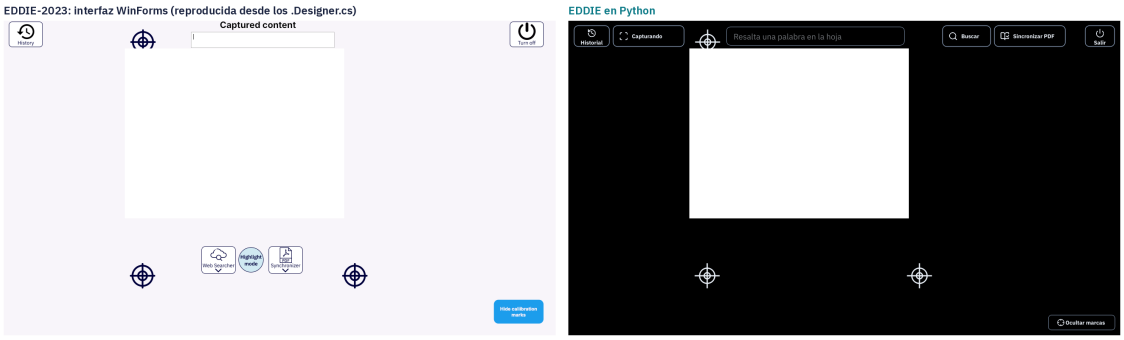


Figura C.3: Pantalla proyectada: EDDIE-2023 (izquierda) y versión en Python (derecha).

Fuente: Elaboración propia.

APÉNDICE D. MANUAL DE INSTALACIÓN Y USO

D.1 INSTALACIÓN

1. Instalar Python 3.12 (64 bits) y, para el reconocimiento de texto, Tesseract-OCR con el idioma español.
2. En la carpeta del proyecto:

   ```
   python -m venv .venv
   .venv\Scripts\activate
   pip install -r requirements.txt
   ```

3. Para las búsquedas, copiar `config/secrets.example.json` como `config/secrets.json` y pegar la clave de Google AI Studio, o definir la variable `GEMINI_API_KEY`. Este archivo no se sube al repositorio.
4. Para el rastreo ocular, abrir GazePoint Control o EyeTribe Server antes de conectar el eye tracker desde la aplicación.

D.2 EJECUCIÓN

```
python main.py              # con el hardware del laboratorio
python main.py --sim        # con cámaras y eye tracker simulados
python -m pytest            # pruebas automatizadas
python tools\generar_evidencias.py   # evidencias
```

D.3 USO

1. Al abrir, la pantalla de inicio muestra los pasos para preparar la sesión y el estado de cada módulo; cada paso lleva a su página.
2. En *Texto* y *Gestos*, elegir la cámara y la resolución. En gestos, elegir el complemento (lápiz, mano o mouse), iniciarlo y calibrar apuntando a la cruz superior izquierda y luego a la inferior derecha.
3. En *Documento PDF*, elegir el PDF del libro.
4. En *Rastreo ocular*, elegir el eye tracker, la retícula y el registro CSV, y pulsar *Aplicar y conectar*.
5. Guardar e iniciar la proyección. En la proyección, resaltar una palabra; el texto reconocido aparece en el campo superior y los botones de búsqueda entregan definición, traducción, enciclopedia, video o imagen. *Sincronizar PDF* abre las herramientas de consistencia papel–PDF.

Los ajustes se guardan en `config/user_settings.json`, las búsquedas en `CSV_registro_actividades/` y cada acción del orquestador con su latencia en `logs/`.